

NEWS RECOMMENDATION USING GRAPH CONVOLUTIONAL NEURAL NETWORK

*MINI Project Report submitted to
Jawaharlal Nehru Technological University Hyderabad
in partial fulfillment for the award of degree of*

Bachelor of Technology

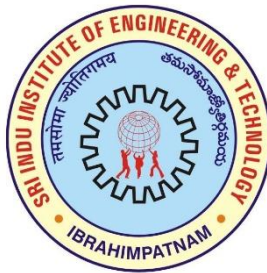
In

Computer Science & Engineering (Data Science)

by

M.SHIVARAM (23X35A6702)
M.NAVYA (22X31A6737)
B. RAKESH (22X31A6710)
A. VARUN TEJA (22X31A6701)

**Under the Guidance of
Dr. P. VEDAVALLI
Associate Professor**



DEPARTMENT OF CSE (DATA SCIENCE)

SRI INDU INSTITUTE OF ENGINEERING & TECHNOLOGY(AUTONOMOUS)

(Affiliated to JNTUH, Hyderabad, Approved by AICTE, New Delhi)

Sheriguda (V), Ibrahimpatnam (M), R.R.Dist., Telangana- 501510.

2025-2026



SRI INDU INSTITUTE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution under UGC)

Accredited by NAAC with A+ Grade, Recognized under 2(f) of UGC Act 1956

(Approved by AICTE, New Delhi and Affiliated to JNTUH, Hyderabad)

Khalsa Ibrahimpatnam, Sheriguda (V), Ibrahimpatnam (M), Ranga Reddy Dist., Telangana – 501 510

Website: <https://siiet.ac.in/>



CERTIFICATE

DEPARTMENT OF CSE (DATA SCIENCE)

This is to certify that the dissertation entitled “**NEWS RECOMMENDATION USING GRAPH CONVOLUTIONAL NEURAL NETWORK**” being submitted by **M. SHIVARAM, M. NAVYA, B. RAKESH, A. VARUNTEJA** bearing Roll No:23X35A6702, 22X31A6737, 22X31A6710, 22X31A6701 to **Jawaharlal Nehru Technological University Hyderabad** in partial fulfillment of the requirements for the award of the degree of ***Bachelor of Technology* in CSE (DATA SCIENCE)**, is a record of bonafide work carried out by him. The results of investigations enclosed in this report have been verified and found satisfactory. The results embodied in this report have not been submitted to any other University or Institute for the award of any other degree

INTERNAL GUIDE

Dr. P. VEDAVALLI

PROJECT CO-ORDINATOR

MS. D. SRUTHI

HEAD OF THE DEPARTMENT

Dr. B. RATNAKANTH

PRINCIPAL

Dr. K. S. SADASIVA RAO

EXTERNAL EXAMINER



SRI INDU INSTITUTE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution under UGC)

Accredited by NAAC with A+ Grade, Recognized under 2(f) of UGC Act 1956

(Approved by AICTE, New Delhi and Affiliated to JNTUH, Hyderabad)

Khalsa Ibrahimpatnam, Sheriguda (V), Ibrahimpatnam (M), Ranga Reddy Dist., Telangana – 501 510

Website: <https://siiet.ac.in/>



DECLARATION

We, **M.SHIVARAM, M.NAVYA, B.RAKESH, A.VARUNTEJA** bearing Roll No: **23X35A6702, 22X31A6737, 22X31A6710, 22X31A6701** hereby certify that the dissertation entitled “**NEWS RECOMMENDATION USING GRAPH CONVOLUTIONAL NEURAL NETWORK**”, carried out under the guidance of **Dr. P.VEDAVALLI Associate Professor** is submitted to **Jawaharlal Nehru Technological University Hyderabad** in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science & Engineering**. This is a record of bonafide work carried out by us and the results embodied in this dissertation have not been reproduced or copied from any source. The results embodied in this dissertation have not been submitted to any other University or Institute for the award of any other degree.

Date:

M.SHIVARAM (23X35A6702)

M.NAVYA (22X31A6737)

B.RAKESH (22X31A6710)

A.VARUNTEJA (22X31A6701)

Department of CSE(DATA SCIENCE),SIJET

ACKNOWLEDGEMENT

With great pleasure we take this opportunity to express our heartfelt gratitude to all the persons who helped us in making this project work a success.

First of all, we express our sincere thanks to **Mr. R. VENKAT RAO, Chairman**, Sri Indu Group of Institutions, for his continuous encouragement.

We are highly indebted to Principal, **Dr. K. S. SADASIVA RAO** for giving us the permission to carry out this project.

We would like to thank **Dr. B. RATNAKANTH**, Professor & Head of the Department (DATA SCIENCE), for giving support through out the period of our study in SIJET. We are grateful for his valuable suggestions and guidance during the execution of this project work.

Our sincere thanks to project guide **Dr. P. VEDAVALLI, Associate Professor** for potentially explaining the entire system and clarifying the queries at every stage of the project.

Our whole hearted thanks to the **Ms. D. Sruthi, Project Co-Ordinator** for her continuous monitoring and Guidance throughout project and we thank staff of CSE(**DS**) who co-operated me for the completion of the project in time.

We also thank our parents and friends who aided us in completion of the project.

M.SHIVARAM (23X35A6702)

M.NAVYA (22X31A6737)

B.RAKESH (22X31A6710)

A.VARUNTEJA (22X31A6701)



SRI INDU INSTITUTE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution under UGC)

Accredited by NAAC with A+ Grade, Recognized under 2(f) of UGC Act 1956

(Approved by AICTE, New Delhi and Affiliated to JNTUH, Hyderabad)

Khalsa Ibrahimpatnam, Sheriguda (V), Ibrahimpatnam (M), Ranga Reddy Dist., Telangana – 501 510

Website: <https://siiet.ac.in/>



INSTITUTE VISION

To become a premier institute of academic excellence by providing the world class education that transforms individuals into high intellectuals, by evolving them as empathetic and responsible citizens through continuous improvement.

INSTITUTE MISSION

- IM1:** To offer outcome-based education and enhancement of technical and practical skills
- IM2:** To continuous assess of teaching-learning process through institute-industry collaboration.
- IM3:** To be a center of excellence for innovative and emerging fields in technology development with state-of-art facilities to faculty and students fraternity.
- IM4:** To create an enterprising environment to ensure culture, ethics and social responsibility among the stakeholders.



SRI INDU INSTITUTE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution under UGC)

Accredited by NAAC with A+ Grade, Recognized under 2(f) of UGC Act 1956

(Approved by AICTE, New Delhi and Affiliated to JNTUH, Hyderabad)

Khalsa Ibrahimpatnam, Sheriguda (V), Ibrahimpatnam (M), Ranga Reddy Dist., Telangana – 501 510

Website: <https://siiet.ac.in/>



Department of CSE (Data Science)

DEPARTMENT VISION

To impart quality education for producing highly talented globally recognizable technocrats and entrepreneurs with ethics, latest knowledge, and innovative ideas in Computer Science and Engineering (Data Science) to meet industrial needs and societal expectations.

DEPARTMENT MISSION

- DM1:** To endow students with innovative and cognitive skills in the field of Data science.
- DM2:** To produce ethical, motivated, and skilled engineers through theoretical knowledge and practical applications
- DM3:** To impart the ability for tackling simple to complex problems individually as well as in a team.
- DM4:** To develop globally competent engineers with strong foundations, capable of “out of the box” thinking so as to adapt to the rapidly changing scenarios requiring socially conscious green computing solutions.

ABSTRACT

In the era of information overload, personalized news recommendation systems have become essential tools to help users navigate vast volumes of digital content. Traditional recommendation approaches, such as collaborative filtering and content-based filtering, often fall short in capturing the complex relationships and contextual information among users, news articles, and their associated metadata. This project proposes a novel news recommendation framework utilizing **Graph Convolutional Neural Networks (GCNs)**, which model the recommendation problem as a graph-based learning task. In this approach, a heterogeneous graph is constructed where nodes represent users, news articles, and entities (e.g., topics, categories, publishers), while edges represent interactions such as user clicks, article content similarities, and category associations. By applying GCNs over this graph, the system is capable of learning rich and high-order node embeddings that effectively capture both the direct and indirect relationships in the data. The embeddings are then used to predict user preferences and recommend the most relevant news articles. The framework is evaluated on real-world datasets such as MIND (Microsoft News Dataset), demonstrating superior performance in terms of accuracy and relevance compared to traditional and deep learning-based baselines. This GCN-based approach not only enhances the personalization of news feeds but also addresses challenges such as data sparsity and cold-start problems, offering a scalable and intelligent solution for next-generation recommender systems.

LIST OF CONTENTS

Topics	Page No
ACKNOWLEDGEMENT	i
INSTITUTE VISION	ii
DEPARTMENT VISION	iii
ABSTRACT	iv
CONTENTS	v
LIST OF FIGURES	viii
1.INTRODUCTION	1
2. LITERATURE SURVEY	2
2.1. Title: Graph Neural Networks for Social Recommendation	2
2.2 Title: Neural News Recommendation with Multi-Head Self-Attention	2
2.3 Title: GCN-based Click-Through Rate Prediction for News Recommendation	2
2.4 Title: A Survey on Deep Learning for Recommender Systems	2
2.5 Title: Knowledge-aware Graph Neural Networks with Label Smoothness Regularization for Recommendation Systems	3
2.6 Existing System	3
2.7 Proposed System	4
2.8 System Study	4
2.8.1 Feasibility Study	4
2.8.2 Economical Feasibility	5
2.8.3 Technical Feasibility	5
2.8.4 Social Feasibility	5
3. SYSTEM ANALYSIS	6
3.1 Introduction	6
3.2 Requirement Specification	6
3.2.1 Functional Requirements	6
3.2.2 Non-Functional Requirements.	6
3.3 System Requirements	7
4. SYSTEM DESIGN	8
4.1 System Architecture	8

4.2 UML Diagrams	8
4.2.1 Class Diagram	8
4.2.2 Use Case Diagram	9
4.2.3 Sequence Diagram	10
4.2.4 Collaborative Diagram	10
4.3 Input and Output Design	11
4.3.1 Input Design	11
4.3.2 Output Design	11
5. IMPLEMENTATIONS	13
5.1 Data Collection and Preprocessing	13
5.2 Graph Construction	13
5.3 Model Development and Training	13
5.4 Recommendation Generation	14
5.5 Integration and Deployment	14
5.6 Maintenance and Scalability	14
5.7 System Environment	14
5.8 Modules Used in Project	25
5.9 Install Python Step-by-Step in Windows and Mac	26
5.10 Sample Code	28
5.11 Results	32
6. SYSTEM TEST	41
6.1 Introduction	41
6.2 TYPES OF TESTS	41
6.2.1 Unit testing	41
6.2.2 Integration Testing	41
6.2.2 Integration Testing	41
6.2.3 Functional Testing	42
6.2.4 System Test	42
6.2.5 White Box Testing	42
6.2.6 Black Box Testing	42
6.2.7 Unit Testing	42
6.2.8 Integration Testing	43
6.2.9 Acceptance Testing	43

7.Conclusion	45
8.Future Enhancements	46
9.References	47

LIST OF FIGURES

Fig 1: System Architecture	8
Fig 2: Class Diagram	8
Fig 3: Use Case Diagram	9
Fig 4: Sequence Diagram	10
Fig.5: Collaborative Diagram	10
Fig 6: Importing Required Python Libraries	32
Fig 7: Loading and Displaying News Dataset	32
Fig 8: Loading and Displaying User Behavior Dataset	33
Fig 9: BERT Model Initialization and Data Extraction Function	33
Fig 10: Data Processing and Training Data Preparation	34
Fig 11: Generated BERT Embedding Vectors	34
Fig 12: Dataset Shuffling and Train-Test Split	35
Fig 13: Performance Metrics Calculation Setup	35
Fig 14: Graph CNN Model Training Process	36
Fig 15: Graph CNN Performance Results (Accuracy & Confusion Matrix)	36
Fig 16: LSTM Model Training Process	37
Fig 17: LSTM Model Performance Results	37
Fig 18: CNN Model Training Process	38
Fig 19: CNN Model Performance Results	38
Fig 20: Performance Comparison Graph of All Algorithms	39
Fig 21: Performance Comparison Table	39
Fig 22: Input Test Data for Recommendation Prediction	40
Fig 23 : Final Recommendation Output (Click Prediction Result)	40

1. INTRODUCTION

In the digital age, users are constantly bombarded with a vast and ever-growing volume of news content from various online sources. As the availability of information increases, so does the challenge of efficiently delivering relevant and personalized content to users. To tackle this problem, news recommendation systems have emerged as critical tools that tailor news delivery to individual user preferences, enhancing user engagement and satisfaction. However, building effective news recommendation systems is far from trivial due to several inherent challenges such as the dynamic nature of news, user preference shifts, data sparsity, and the cold-start problem for new users and articles.

Traditional recommendation techniques, including collaborative filtering, content-based filtering, and hybrid models, have been widely used in commercial systems. While these methods offer baseline performance, they often suffer from significant limitations. Collaborative filtering relies heavily on user-item interaction histories and struggles with sparse data or new users/items. Content-based methods focus solely on item features, often leading to limited diversity and inability to leverage social or contextual information.

To overcome these challenges, recent advancements in graph-based deep learning have opened up new possibilities for recommendation systems. Graph Convolutional Neural Networks (GCNs), a class of neural networks designed to operate on graph-structured data, provide a powerful mechanism to model complex relationships and higher-order connections between entities. In the context of news recommendation, user behaviors, news content, and metadata can naturally be represented as a heterogeneous graph, where nodes represent users, articles, categories, or topics, and edges capture various types of interactions and similarities.

This project proposes a GCN-powered news recommendation system that effectively integrates multiple sources of information by modeling them in a unified graph structure. The system first constructs a heterogeneous graph connecting users to the news they have read, articles to their categories and entities, and even inter-article similarities based on textual content. By applying GCN layers to this graph, we can propagate features and learn deep representations for each node that encapsulate not just direct interactions but also global contextual information through multi-hop neighbor aggregation.

The learned embeddings are then used to predict user-article interaction scores, enabling accurate ranking and personalized recommendation of news articles. The proposed approach is evaluated on benchmark datasets such as MIND (Microsoft News Dataset), demonstrating notable improvements in key metrics like precision, recall, and NDCG compared to traditional baselines and deep learning models.

2. LITERATURE SURVEY

2.1. Title: Graph Neural Networks for Social Recommendation

This paper introduces Graph Rec, a framework that models user-item interactions as a bipartite graph and applies graph neural networks to learn representations for recommendation. The model incorporates user opinions and social influences using graph convolutions, showing that GNNs can effectively capture user behavior and preferences in a recommendation context. Although it focuses on social networks, the approach is relevant for news recommendation by highlighting the potential of GNNs to model relationships beyond direct interactions.

2.2 Title: Neural News Recommendation with Multi-Head Self-Attention

This study proposes the NRMS (Neural News Recommendation with Multi-Head Self-Attention) model, which encodes news titles using self-attention and models user interests by aggregating news clicked by the user. While not graph-based, NRMS sets a strong baseline in news recommendation using deep learning, and its techniques for modeling news semantics can be integrated into GCN-based models for better node feature representations.

2.3 Title: GCN-based Click-Through Rate Prediction for News Recommendation

This work applies Graph Convolutional Networks to improve click-through rate prediction in news recommendation. The authors construct a heterogeneous graph from user-news interactions and train a GCN to learn embeddings for better recommendation performance. This research is closely aligned with the current project and demonstrates the advantages of GCNs in handling sparse data and capturing complex interactions in a news recommendation environment.

2.4 Title: A Survey on Deep Learning for Recommender Systems

This comprehensive survey discusses various deep learning architectures, including convolutional neural networks, recurrent networks, and graph neural networks, in the context of recommendation systems. It highlights the strengths and limitations of each approach and underlines how GCNs are particularly effective in capturing non-Euclidean structures in user-item interaction data.

2.5 Title: Knowledge-aware Graph Neural Networks with Label Smoothness Regularization for Recommendation Systems

This paper presents a knowledge-aware GCN model that incorporates external knowledge graphs into the recommendation process. The model applies GCNs with label smoothness regularization to improve item representations and user interest modeling. This work supports the idea that enriching the graph structure with semantic or categorical information can significantly enhance recommendation accuracy in systems like news recommendation.

2.6 Existing System:

Existing news recommendation systems mainly use collaborative filtering, content-based filtering, and deep learning methods. Collaborative filtering recommends articles based on similar users' behavior but depends heavily on past interactions, leading to the cold-start problem for new users and articles. Content-based filtering suggests articles based on previously read content using techniques like TF-IDF, Word2Vec, and BERT, but it often lacks diversity and ignores broader user trends. To address these issues, deep learning models such as CNNs, RNNs, and attention-based models like NRMS have been introduced to improve personalization and semantic understanding. However, these models still represent user-item interactions in simple forms like sequences or matrices, failing to capture complex relationships among users, articles, and topics, which limits their overall effectiveness. Additionally, these systems often struggle to adapt quickly to rapidly changing news trends and user preferences. They may also require large volumes of data and high computational resources, making real-time recommendations more challenging. These limitations highlight the need for more advanced and adaptive recommendation approaches.

Disadvantages of Existing Systems:

1. Cold-Start Problem
2. Data Sparsity
3. Lack of Diversity
4. Inability to Capture Complex Relationships
5. Limited Context Awareness

2.7 Proposed System:

The proposed system introduces an advanced news recommendation approach using Graph Convolutional Neural Networks (GCNs), which overcome the limitations of traditional methods that rely on flat user-item interactions. It constructs a heterogeneous graph where users, news articles, categories, keywords, and metadata are represented as nodes, and their relationships—such as user clicks, topic associations, and keyword similarities—are modeled as edges. This graph-based structure enables the system to capture complex, high-order relationships and dependencies that are not possible with linear or sequence-based models. By representing interactions in this interconnected form, the system gains a deeper understanding of both user behavior and content characteristics.

To extract meaningful insights, graph convolution operations aggregate information from neighboring nodes, allowing each node to learn from both its own features and its surrounding context. This process generates rich embeddings for users and articles, improving recommendation accuracy. The system effectively reduces the cold-start problem by leveraging indirect relationships such as shared categories or keywords, even when direct interaction data is limited. It also addresses data sparsity by utilizing the graph structure to infer missing information. Additionally, the model enhances diversity by exploring a wider range of related content, resulting in more personalized, varied, and engaging news recommendations for users.

Advantages of the Proposed System:

1. Captures Complex Relationships
2. Solves Cold-Start and Data Sparsity Issues
3. Enhances Diversity and Novelty
4. Improves Context Awareness
5. Scalable and Real-Time Adaptability
6. Provides Better Personalization

2.8 SYSTEM STUDY

2.8.1 FEASIBILITY STUDY

The feasibility of the project is analyzed in this phase and business proposal is put forth with a very general plan for the project and some cost estimates. During system analysis the feasibility study of the proposed system is to be carried out. This is to ensure that the proposed system is not a burden to the company. For feasibility analysis, some understanding of the major requirements for the system is essential.

Three key considerations involved in the feasibility analysis are

- ECONOMICAL FEASIBILITY
- TECHNICAL FEASIBILITY
- SOCIAL FEASIBILITY

2.8.2 ECONOMICAL FEASIBILITY

This study is carried out to check the economic impact that the system will have on the organization. The amount of fund that the company can pour into the research and development of the system is limited. The expenditures must be justified. Thus the developed system as well within the budget and this was achieved because most of the technologies used are freely available. Only the customized products had to be purchased.

2.8.3 TECHNICAL FEASIBILITY

This study is carried out to check the technical feasibility, that is, the technical requirements of the system. Any system developed must not have a high demand on the available technical resources. This will lead to high demands on the available technical resources. This will lead to high demands being placed on the client. The developed system must have a modest requirement, as only minimal or null changes are required for implementing this system.

2.8.4 SOCIAL FEASIBILITY

The aspect of study is to check the level of acceptance of the system by the user. This includes the process of training the user to use the system efficiently. The user must not feel threatened by the system, instead must accept it as a necessity. The level of acceptance by the users solely depends on the methods that are employed to educate the user about the system and to make him familiar with it. His level of confidence must be raised so that he is also able to make some constructive criticism, which is welcomed, as he is the final user of the system.

3. SYSTEM ANALYSIS

3.1 INTRODUCTION

System Analysis is a critical phase in software development that focuses on understanding the current system, identifying its limitations, and proposing an improved solution. It involves studying user requirements, analyzing existing processes, and defining system functionalities to ensure efficient system design and implementation.

In this project, the system analysis is carried out for a news recommendation system that aims to provide personalized news content to users. The analysis identifies the drawbacks of traditional recommendation techniques and proposes an advanced solution using Graph Convolutional Neural Networks (GCNs).

The goal of this phase is to:

- Understand the current recommendation methods
- Identify existing problems such as data sparsity and cold-start
- Define system requirements (functional & non-functional)
- Propose an efficient and scalable solution

3.2 REQUIREMENT SPECIFICATION

3.2.1 Functional Requirements

- User Registration & Login: System should allow users to create accounts and log in securely.
- News Data Collection: System should collect news articles from datasets or APIs.
- Data Preprocessing: System should clean and preprocess news data (remove noise, normalize text).
- Feature Extraction: System should convert news articles into embeddings using models like BERT.
- Graph Construction: System should build a graph connecting users, news, and metadata.
- Model Training: System should train the GCN model using user interaction data.
- Recommendation Generation: System should generate personalized news recommendations for users.
- User Interaction Tracking: System should track clicks, views, and user behavior.
- Result Display: System should display recommended news articles to users.

3.2.2 Non-Functional Requirements

- Performance: System should provide fast and real-time recommendations.
- Scalability: System should handle large datasets and increasing users efficiently.
- Accuracy: Recommendations should be highly relevant and personalized.

- Reliability: System should function without failure during continuous operation.
- Usability: Interface should be simple, user-friendly, and easy to navigate.
- Security: User data should be protected and securely stored.
- Maintainability: System should be easy to update and modify.
- Flexibility: System should adapt to new data and changing user preferences.
- Availability: System should be accessible at all times.
- Efficiency: System should optimize resource usage (memory, CPU).

3.3 SYSTEM REQUIREMENTS:

HARDWARE REQUIREMENTS:

- System : Pentium IV 2.4 GHz.
- Hard Disk : 40 GB.
- Ram : 512 Mb.

SOFTWARE REQUIREMENTS:

- Operating system : - Windows.
- Coding Language : python.

4. SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

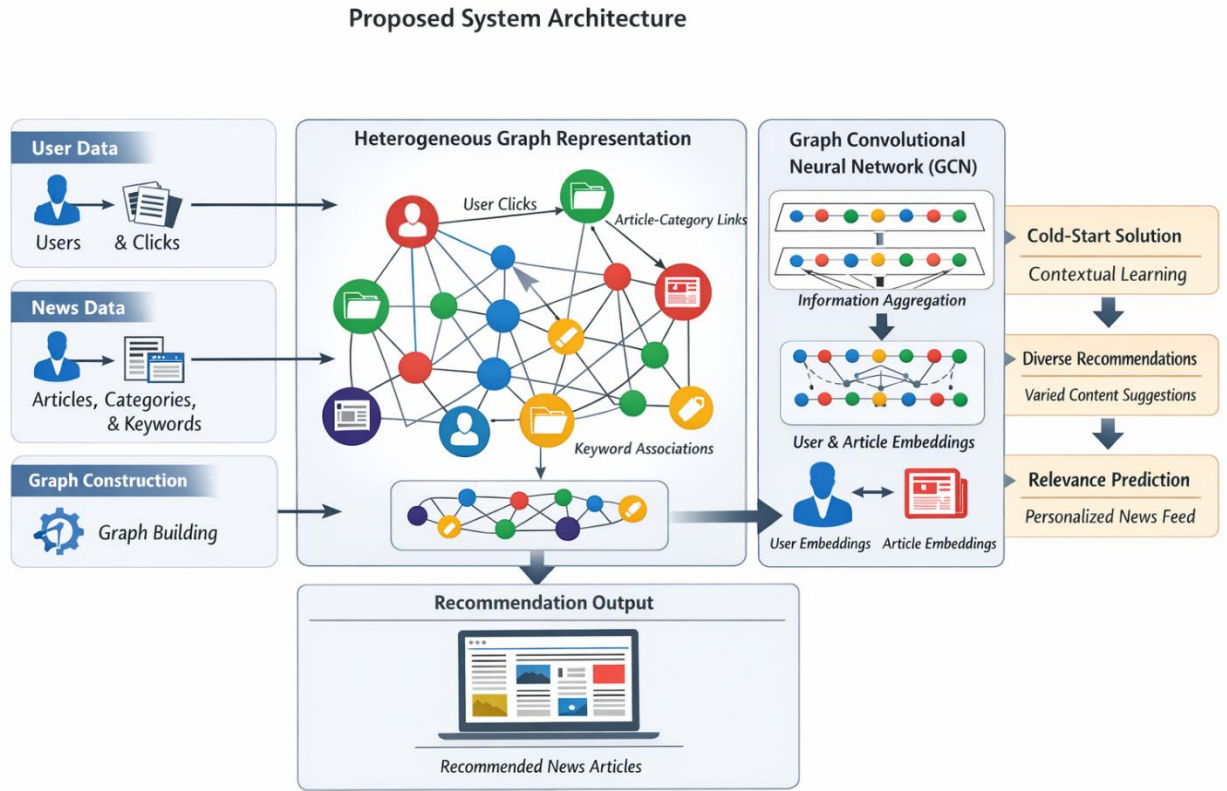


Fig 1: System Architecture

4.2 UML DIAGRAMS

4.2.1 CLASS DIAGRAM

The class diagram is used to refine the use case diagram and define a detailed design of the system. The class diagram classifies the actors defined in the use case diagram into a set of interrelated classes. The relationship or association between the classes can be either an "is-a" or "has-a" relationship. Each class in the class diagram may be capable of providing certain functionalities. These functionalities provided by the class are termed "methods" of the class. Apart from this, each class may have certain "attributes" that uniquely.

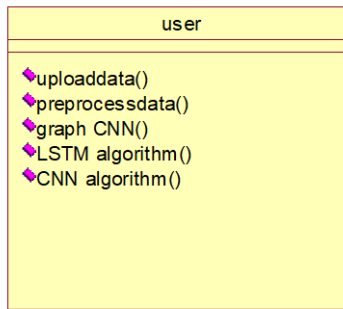


Fig 2: Class Diagram

4.2.2 USE CASE DIAGRAM

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.

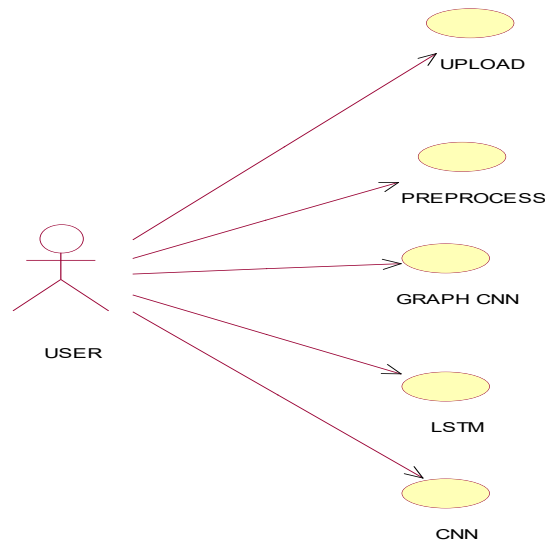


Fig 3: Use Case Diagram

4.2.3 SEQUENCE DIAGRAM

A sequence diagram represents the interaction between different objects in the system. The important aspect of a sequence diagram is that it is time-ordered. This means that the exact sequence of the interactions between the objects is represented step by step. Different objects in the sequence diagram interact with each other by passing "messages".

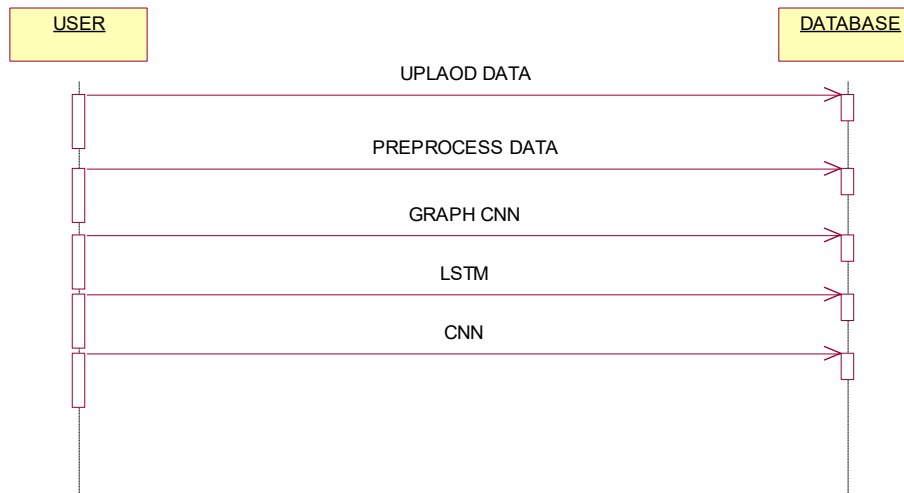


Fig 4: Sequence Diagram

4.2.4 COLLABORATIVE DIAGRAM

A collaboration diagram groups together the interactions between different objects. The interactions are listed as numbered interactions that help to trace the sequence of the interactions. The collaboration diagram helps to identify all the possible interactions that each object has with other objects.

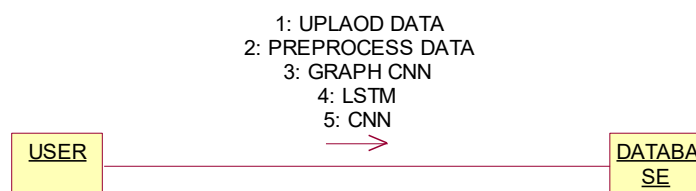


Fig.5: Collaborative Diagram

4.3 INPUT AND OUTPUT DESIGN

4.3.1 INPUT DESIGN

The input design is the link between the information system and the user. It comprises the developing specification and procedures for data preparation and those steps are necessary to put transaction data in to a usable form for processing can be achieved by inspecting the computer to read data from a written or printed document or it can occur by having people keying the data directly into the system. The design of input focuses on controlling the amount of input required, controlling the errors, avoiding delay, avoiding extra steps and keeping the process simple. The input is designed in such a way so that it provides security and ease of use with retaining the privacy. Input Design considered the following things:

- What data should be given as input?
- How the data should be arranged or coded?
- The dialog to guide the operating personnel in providing input.
- Methods for preparing input validations and steps to follow when error occur.

OBJECTIVES

1. Input Design is the process of converting a user-oriented description of the input into a computer-based system. This design is important to avoid errors in the data input process and show the correct direction to the management for getting correct information from the computerized system.
2. It is achieved by creating user-friendly screens for the data entry to handle large volume of data. The goal of designing input is to make data entry easier and to be free from errors. The data entry screen is designed in such a way that all the data manipulates can be performed. It also provides record viewing facilities.
3. When the data is entered it will check for its validity. Data can be entered with the help of screens. Appropriate messages are provided as when needed so that the user will not be in maize of instant. Thus the objective of input design is to create an input layout that is easy to follow

4.3.2 OUTPUT DESIGN

A quality output is one, which meets the requirements of the end user and presents the information clearly. In any system results of processing are communicated to the users and to other system through outputs. In output design it is determined how the information is to be displaced for immediate need and also the hard copy output. It is the most important and direct source information to the user. Efficient and intelligent output design improves the system's relationship to help user decision-making.

1. Designing computer output should proceed in an organized, well thought out manner; the right output must be developed while ensuring that each output element is designed so that people will find the system can use

easily and effectively. When analysis design computer output, they should Identify the specific output that is needed to meet the requirements.

2.Select methods for presenting information.

3.Create document, report, or other formats that contain information produced by the system.

The output form of an information system should accomplish one or more of the following objectives.

- Convey information about past activities, current status or projections of the
- Future.
- Signal important events, opportunities, problems, or warnings.
- Trigger an action.
- Confirm an action.

5. IMPLEMENTATIONS

5.1 Data Collection and Preprocessing

The implementation begins with gathering data from various sources, including user interactions such as clicks, reads, and likes on news articles, along with article metadata like categories, keywords, and publication timestamps. This raw data is cleaned and normalized to remove noise and inconsistencies. Features are extracted and encoded appropriately to serve as inputs for the graph nodes, ensuring the data is ready for graph construction and model training.

5.2 Graph Construction

A heterogeneous graph is constructed where nodes represent users, news articles, and related entities like categories and keywords. Edges in the graph capture different relationships such as user-article interactions, article-category associations, and similarity between articles based on shared keywords. This graph structure forms the foundation for applying Graph Convolutional Neural Networks, enabling the system to capture complex interconnections in the data.

5.3 Model Development and Training

Using graph neural network libraries such as PyTorch Geometric or Deep Graph Library (DGL), the Graph Convolutional Neural Network layers are implemented to perform neighborhood aggregation and feature learning. The model is trained by feeding the graph and node features into the GCN, optimizing with loss functions suited for recommendation, such as Bayesian Personalized Ranking (BPR) or cross-entropy loss. Techniques like early stopping and cross-validation are applied to ensure robust training and prevent overfitting.

5.4 Recommendation Generation

After training, the GCN model produces embeddings that capture user preferences and article features. These embeddings are used to compute relevance scores between users and news articles. Based on these scores, personalized ranked lists of news recommendations are generated for each user. The system supports updating recommendations in near real-time by incorporating new data and retraining or incrementally updating the model.

5.5 Integration and Deployment

The trained recommendation engine is integrated with the news platform through API endpoints or user interfaces, delivering personalized news feeds seamlessly to users. Logging and monitoring components track system performance, recommendation accuracy, and user engagement metrics. Security and privacy measures, including data anonymization and secure storage, are implemented to comply with relevant regulations and protect user data.

5.6 Maintenance and Scalability

The system is designed for scalability to handle growing numbers of users and articles efficiently. Periodic retraining and incremental learning approaches allow the model to adapt to changing user behavior and newly published content without full retraining. Maintenance routines include monitoring system health, updating graph data, and refining model parameters to sustain high recommendation quality

5.7 SYSTEM ENVIRONMENT:

What is Python :-

Below are some facts about Python.

Python is currently the most widely used multi-purpose, high-level programming language.

Python allows programming in Object-Oriented and Procedural paradigms. Python programs generally are smaller than other programming languages like Java.

Programmers have to type relatively less and indentation requirement of the language, makes them readable all the time.

Python language is being used by almost all tech-giant companies like – Google, Amazon, Facebook, Instagram, Dropbox, Uber... etc.

The biggest strength of Python is huge collection of standard library which can be used for the following .

- Machine Learning
- Applications (like Kivy, Tkinter, PyQt etc.)
- Web frameworks like Django (used by YouTube, Instagram, Dropbox)
- Image processing (like Opencv, Pillow)
- Web scraping (like Scrappy, BeautifulSoup, Selenium)
- Test frameworks
- Multimedia

Advantages of Python :-

1. Extensive Libraries

Python downloads with an extensive library and it contains code for various purposes like regular expressions, documentation-generation, unit-testing, web browsers, threading, databases, CGI, email, image manipulation, and more. So, we don't have to write the complete code for that manually.

2. Extensible

As we have seen earlier, Python can be extended to other languages. You can write some of your code in languages like C++ or C. This comes in handy, especially in projects.

3. Embeddable

Complimentary to extensibility, Python is embeddable as well. You can put your Python code in your source code of a different language, like C++. This lets us add scripting capabilities to our code in the other language.

4. Improved Productivity

The language's simplicity and extensive libraries render programmers more productive than languages like Java and C++ do. Also, the fact that you need to write less and get more things done.

5. IOT Opportunities

Since Python forms the basis of new platforms like Raspberry Pi, it finds the future bright for the Internet Of Things. This is a way to connect the language with the real world.

6. Simple and Easy

When working with Java, you may have to create a class to print 'Hello World'. But in Python, just a print statement will do. It is also quite easy to learn, understand, and code. This is why when people pick up Python, they have a hard time adjusting to other more verbose languages like Java.

7. Readable

Because it is not such a verbose language, reading Python is much like reading English. This is the reason why it is so easy to learn, understand, and code. It also does not need curly braces to define blocks, and indentation is mandatory. This further aids the readability of the code.

8. Object-Oriented

This language supports both the procedural and object-oriented programming paradigms. While functions help us with code reusability, classes and objects let us model the real world. A class allows the encapsulation of data and functions into one.

9. Free and Open-Source

Like we said earlier, Python is freely available. But not only can you download Python for free, but you can also download its source code, make changes to it, and even distribute it. It downloads with an extensive collection of libraries to help you with your tasks.

10. Portable

When you code your project in a language like C++, you may need to make some changes to it if you want to run it on another platform. But it isn't the same with Python. Here, you need to code only once, and you can run it anywhere. This is called Write Once Run Anywhere (WORA). However, you need to be careful enough not to include any system-dependent features.

11. Interpreted

Lastly, we will say that it is an interpreted language. Since statements are executed one by one, debugging is easier than in compiled languages.

Advantages of Python Over Other Languages

1. Less Coding

Almost all of the tasks done in Python requires less coding when the same task is done in other languages. Python also has an awesome standard library support, so you don't have to search for any third-party libraries to get your job done. This is the reason that many people suggest learning Python to beginners.

2. Affordable

Python is free therefore individuals, small companies or big organizations can leverage the free available resources to build applications. Python is popular and widely used so it gives you better community support.

The 2019 Github annual survey showed us that Python has overtaken Java in the most popular programming language category.

3. Python is for Everyone

Python code can run on any machine whether it is Linux, Mac or Windows. Programmers need to learn different languages for different jobs but with Python, you can professionally build web apps, perform data analysis and machine learning, automate things, do web scraping and also build games and powerful visualizations. It is an all-rounder programming language.

Disadvantages of Python

So far, we've seen why Python is a great choice for your project. But if you choose it, you should be aware of its consequences as well. Let's now see the downsides of choosing Python over another language.

1. Speed Limitations

We have seen that Python code is executed line by line. But since Python is interpreted, it often results in slow execution. This, however, isn't a problem unless speed is a focal point for the project. In other words, unless high speed is a requirement, the benefits offered by Python are enough to distract us from its speed limitations.

2. Weak in Mobile Computing and Browsers

While it serves as an excellent server-side language, Python is much rarely seen on the client-side. Besides that, it is rarely ever used to implement smartphone-based applications. One such application is called Carbonnelle.

The reason it is not so famous despite the existence of Brython is that it isn't that secure.

3. Design Restrictions

As you know, Python is dynamically-typed. This means that you don't need to declare the type of variable while writing the code. It uses duck-typing. But wait, what's that? Well, it just means that if it looks like a duck, it must be a duck. While this is easy on the programmers during coding, it can raise run-time errors.

4. Underdeveloped Database Access Layers

Compared to more widely used technologies like JDBC (Java DataBase Connectivity) and ODBC (Open DataBase Connectivity), Python's database access layers are a bit underdeveloped. Consequently, it is less often applied in huge enterprises.

5. Simple

No, we're not kidding. Python's simplicity can indeed be a problem. Take my example. I don't do Java, I'm more of a Python person. To me, its syntax is so simple that the verbosity of Java code seems unnecessary. This was all about the Advantages and Disadvantages of Python Programming Language.

History of Python :-

What do the alphabet and the programming language Python have in common? Right, both start with ABC. If we are talking about ABC in the Python context, it's clear that the programming language ABC is meant. ABC is a general-purpose programming language and programming environment, which had been developed in the Netherlands, Amsterdam, at the CWI (Centrum Wiskunde & Informatica). The greatest achievement of ABC was to influence the design of Python. Python was conceptualized in the late 1980s. Guido van Rossum worked that time in a project at the CWI, called Amoeba, a distributed operating system. In an interview with Bill Venners¹, Guido van Rossum said: "In the early 1980s, I worked as an implementer on a team building a language called ABC at Centrum voor Wiskunde en Informatica (CWI).

I don't know how well people know ABC's influence on Python. I try to mention ABC's influence because I'm indebted to everything I learned during that project and to the people who worked on it." Later on in the same Interview, Guido van Rossum continued: "I remembered all my experience and some of my frustration with ABC. I decided to try to design a simple scripting language that possessed some of ABC's better properties, but without its problems. So I started typing. I created a simple virtual machine, a simple parser, and a simple runtime. I made my own version of the various ABC parts that I liked. I created a basic syntax, used indentation

for statement grouping instead of curly braces or begin-end blocks, and developed a small number of powerful data types: a hash table (or dictionary, as we call it), a list, strings, and numbers."

What is Machine Learning : -

Before we take a look at the details of various machine learning methods, let's start by looking at what machine learning is, and what it isn't. Machine learning is often categorized as a subfield of artificial intelligence, but I find that categorization can often be misleading at first brush. The study of machine learning certainly arose from research in this context, but in the data science application of machine learning methods, it's more helpful to think of machine learning as a means of *building models of data*.

Fundamentally, machine learning involves building mathematical models to help understand data. "Learning" enters the fray when we give these models *tunable parameters* that can be adapted to observed data; in this way the program can be considered to be "learning" from the data.

Once these models have been fit to previously seen data, they can be used to predict and understand aspects of newly observed data. I'll leave to the reader the more philosophical digression regarding the extent to which this type of mathematical, model-based "learning" is similar to the "learning" exhibited by the human brain. Understanding the problem setting in machine learning is essential to using these tools effectively, and so we will start with some broad categorizations of the types of approaches we'll discuss here.

Categories Of Machine Learning :

At the most fundamental level, machine learning can be categorized into two main types: supervised learning and unsupervised learning.

Supervised learning involves somehow modeling the relationship between measured features of data and some label associated with the data; once this model is determined, it can be used to apply labels to new, unknown data. This is further subdivided into *classification* tasks and *regression* tasks: in classification, the labels are discrete categories, while in regression, the labels are continuous quantities. We will see examples of both types of supervised learning in the following section.

Unsupervised learning involves modeling the features of a dataset without reference to any label, and is often described as "letting the dataset speak for itself." These models include tasks such as clustering and dimensionality reduction.

Clustering algorithms identify distinct groups of data, while dimensionality reduction algorithms search for more succinct representations of the data. We will see examples of both types of unsupervised learning in the following section.

Need for Machine Learning

Human beings, at this moment, are the most intelligent and advanced species on earth because they can think, evaluate and solve complex problems. On the other side, AI is still in its initial stage and haven't surpassed human intelligence in many aspects. Then the question is that what is the need to make machine learn? The most suitable reason for doing this is, "to make decisions, based on data, with efficiency and scale".

Lately, organizations are investing heavily in newer technologies like Artificial Intelligence, Machine Learning and Deep Learning to get the key information from data to perform several real-world tasks and solve problems. We can call it data-driven decisions taken by machines, particularly to automate the process. These data-driven decisions can be used, instead of using programming logic, in the problems that cannot be programmed inherently. The fact is that we can't do without human intelligence, but other aspect is that we all need to solve real-world problems with efficiency at a huge scale. That is why the need for machine learning arises.

Challenges in Machines Learning :-

While Machine Learning is rapidly evolving, making significant strides with cybersecurity and autonomous cars, this segment of AI as whole still has a long way to go. The reason behind is that ML has not been able to overcome number of challenges. The challenges that ML is facing currently are –

Quality of data – Having good-quality data for ML algorithms is one of the biggest challenges. Use of low-quality data leads to the problems related to data preprocessing and feature extraction.

Time-Consuming task – Another challenge faced by ML models is the consumption of time especially for data acquisition, feature extraction and retrieval.

Lack of specialist persons – As ML technology is still in its infancy stage, availability of expert resources is a tough job.

No clear objective for formulating business problems – Having no clear objective and well-defined goal for business problems is another key challenge for ML because this technology is not that mature yet.

Issue of overfitting & underfitting – If the model is overfitting or underfitting, it cannot be represented well for the problem.

Curse of dimensionality – Another challenge ML model faces is too many features of data points. This can be a real hindrance.

Difficulty in deployment – Complexity of the ML model makes it quite difficult to be deployed in real life.

Applications of Machines Learning :-

Machine Learning is the most rapidly growing technology and according to researchers we are in the golden year of AI and ML. It is used to solve many real-world complex problems which cannot be solved with traditional approach. Following are some real-world applications of ML –

- Emotion analysis
- Sentiment analysis
- Error detection and prevention
- Weather forecasting and prediction
- Stock market analysis and forecasting
- Speech synthesis
- Speech recognition
- Customer segmentation
- Object recognition
- Fraud detection
- Fraud prevention
- Recommendation of products to customer in online shopping

How to start learning ML?

This is a rough roadmap you can follow on your way to becoming an insanely talented Machine Learning Engineer. Of course, you can always modify the steps according to your needs to reach your desired end-goal!

Step 1 – Understand the Prerequisites

In case you are a genius, you could start ML directly but normally, there are some prerequisites that you need to know which include Linear Algebra, Multivariate Calculus, Statistics, and Python. And if you don't know these, never fear! You don't need a Ph.D. degree in these topics to get started but you do need a basic understanding.

(a) Learn Linear Algebra and Multivariate Calculus

Both Linear Algebra and Multivariate Calculus are important in Machine Learning. However, the extent to which you need them depends on your role as a data scientist. If you are more focused on application heavy machine learning, then you will not be that heavily focused on maths as there are many common libraries available. But if you want to focus on R&D in Machine Learning, then mastery of Linear Algebra and Multivariate Calculus is very important as you will have to implement many ML algorithms from scratch.

(b) Learn Statistics

Data plays a huge role in Machine Learning. In fact, around 80% of your time as an ML expert will be spent collecting and cleaning data. And statistics is a field that handles the collection, analysis, and presentation of data. So it is no surprise that you need to learn it!!! Some of the key concepts in statistics that are important are Statistical Significance, Probability Distributions, Hypothesis Testing, Regression, etc. Also, Bayesian Thinking is also a very important part of ML which deals with various concepts like Conditional Probability, Priors, and Posteriors, Maximum Likelihood, etc.

(c) Learn Python

Some people prefer to skip Linear Algebra, Multivariate Calculus and Statistics and learn them as they go along with trial and error. But the one thing that you absolutely cannot skip is Python! While there are other languages you can use for Machine Learning like R, Scala, etc. Python is currently the most popular language for ML. In fact, there are many Python libraries that are specifically useful for Artificial Intelligence and Machine Learning such as Keras, TensorFlow, Scikit-learn, etc.

So if you want to learn ML, it's best if you learn Python! You can do that using various online resources and courses such as **Fork Python** available Free on Geeks for Geeks.

Step 2 – Learn Various ML Concepts

Now that you are done with the prerequisites, you can move on to actually learning ML (Which is the fun part!!!) It's best to start with the basics and then move on to the more complicated stuff. Some of the basic concepts in ML are:

(a) Terminologies of Machine Learning

- **Model** – A model is a specific representation learned from data by applying some machine learning algorithm. A model is also called a hypothesis.
- **Feature** – A feature is an individual measurable property of the data. A set of numeric features can be conveniently described by a feature vector. Feature vectors are fed as input to the model. For example, in order to predict a fruit, there may be features like color, smell, taste, etc.
- **Target (Label)** – A target variable or label is the value to be predicted by our model. For the fruit example discussed in the feature section, the label with each set of input would be the name of the fruit like apple, orange, banana, etc.
- **Training** – The idea is to give a set of inputs(features) and it's expected outputs(labels), so after training, we will have a model (hypothesis) that will then map new data to one of the categories trained on.
- **Prediction** – Once our model is ready, it can be fed a set of inputs to which it will provide a predicted output(label).

(b) Types of Machine Learning

- **Supervised Learning** – This involves learning from a training dataset with labeled data using classification and regression models. This learning process continues until the required level of performance is achieved.
- **Unsupervised Learning** – This involves using unlabelled data and then finding the underlying structure in the data in order to learn more and more about the data itself using factor and cluster analysis models.
- **Semi-supervised Learning** – This involves using unlabelled data like Unsupervised Learning with a small amount of labeled data. Using labeled data vastly increases the learning accuracy and is also more cost-effective than Supervised Learning.
- **Reinforcement Learning** – This involves learning optimal actions through trial and error. So the next action is decided by learning behaviors that are based on the current state and that will maximize the reward in the future.

Advantages of Machine learning :-

1. Easily identifies trends and patterns -

Machine Learning can review large volumes of data and discover specific trends and patterns that would not be apparent to humans. For instance, for an e-commerce website like Amazon, it serves to understand the browsing behaviors and purchase histories of its users to help cater to the right products, deals, and reminders relevant to them. It uses the results to reveal relevant advertisements to them.

2. No human intervention needed (automation)

With ML, you don't need to babysit your project every step of the way. Since it means giving machines the ability to learn, it lets them make predictions and also improve the algorithms on their own. A common example of this is anti-virus softwares; they learn to filter new threats as they are recognized. ML is also good at recognizing spam.

3. Continuous Improvement

As ML algorithms gain experience, they keep improving in accuracy and efficiency. This lets them make better decisions. Say you need to make a weather forecast model. As the amount of data you have keeps growing, your algorithms learn to make more accurate predictions faster.

4. Handling multi-dimensional and multi-variety data

Machine Learning algorithms are good at handling data that are multi-dimensional and multi-variety, and they can do this in dynamic or uncertain environments.

5. Wide Applications

You could be an e-tailer or a healthcare provider and make ML work for you. Where it does apply, it holds the capability to help deliver a much more personal experience to customers while also targeting the right customers.

Disadvantages of Machine Learning :-

1. Data Acquisition

Machine Learning requires massive data sets to train on, and these should be inclusive/unbiased, and of good quality. There can also be times where they must wait for new data to be generated.

2. Time and Resources

ML needs enough time to let the algorithms learn and develop enough to fulfill their purpose with a considerable amount of accuracy and relevancy. It also needs massive resources to function. This can mean additional requirements of computer power for you.

3. Interpretation of Results

Another major challenge is the ability to accurately interpret results generated by the algorithms. You must also carefully choose the algorithms for your purpose.

4. High error-susceptibility

Machine Learning is autonomous but highly susceptible to errors. Suppose you train an algorithm with data sets small enough to not be inclusive. You end up with biased predictions coming from a biased training set. This leads to irrelevant advertisements being displayed to customers. In the case of ML, such blunders can set off a chain of errors that can go undetected for long periods of time. And when they do get noticed, it takes quite some time to recognize the source of the issue, and even longer to correct it.

Python Development Steps : -

Guido Van Rossum published the first version of Python code (version 0.9.0) at alt. sources in February 1991. This release included already exception handling, functions, and the core data types of list, dict, str and others. It was also object oriented and had a module system .Python version 1.0 was released in January 1994. The major new features included in this release were the functional programming tools lambda, map, filter and reduce, which Guido Van Rossum never liked. Six and a half years later in October 2000, Python 2.0 was introduced. This release included list comprehensions, a full garbage collector and it was supporting unicode. Python flourished for another 8 years in the versions 2.x before the next major release as Python 3.0 (also known as "Python 3000" and "Py3K") was released. Python 3 is not backwards compatible with Python 2.x.

The emphasis in Python 3 had been on the removal of duplicate programming constructs and modules, thus fulfilling or coming close to fulfilling the 13th law of the Zen of Python: "There should be one -- and preferably only one -- obvious way to do it. " Some changes in Python 7.3:

- Print is now a function
- Views and iterators instead of lists
- The rules for ordering comparisons have been simplified. E.g. a heterogeneous list cannot be sorted, because all the elements of a list must be comparable to each other.
- There is only one integer type left, i.e. int. long is int as well.
- The division of two integers returns a float instead of an integer. "/" can be used to have the "old" behaviour.
- Text Vs. Data Instead Of Unicode Vs. 8-bit

Purpose :-

We demonstrated that our approach enables successful segmentation of intra-retinal layers—even with low-quality images containing speckle noise, low contrast, and different intensity ranges throughout—with the assistance of the ANIS feature.

Python

Python is an interpreted high-level programming language for general-purpose programming. Created by Guido van Rossum and first released in 1991, Python has a design philosophy that emphasizes code readability, notably using significant whitespace.

Python features a dynamic type system and automatic memory management. It supports multiple programming paradigms, including object-oriented, imperative, functional and procedural, and has a large and comprehensive standard library.

- Python is Interpreted – Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- Python is Interactive – you can actually sit at a Python prompt and interact with the interpreter directly to write your programs.

Python also acknowledges that speed of development is important. Readable and terse code is part of this, and so is access to powerful constructs that avoid tedious repetition of code. Maintainability also ties into this may be an all but useless metric, but it does say something about how much code you have to scan, read and/or understand to troubleshoot problems or tweak behaviors. This speed of development, the ease with which a programmer of other languages can pick up basic Python skills and the huge standard library is key to another area where Python excels. All its tools have been quick to implement, saved a lot of time, and several of them have later been patched and updated by people with no Python background - without breaking.

5.8 Modules Used in Project :-

Tensorflow

TensorFlow is a free and open-source software library for dataflow and differentiable programming across a range of tasks. It is a symbolic math library, and is also used for machine learning applications such as neural networks. It is used for both research and production at Google.

TensorFlow was developed by the Google Brain team for internal Google use. It was released under the Apache 2.0 open-source license on November 9, 2015.

Numpy

Numpy is a general-purpose array-processing package. It provides a high-performance multidimensional array object, and tools for working with these arrays.

It is the fundamental package for scientific computing with Python. It contains various features including these important ones:

- A powerful N-dimensional array object
- Sophisticated (broadcasting) functions
- Tools for integrating C/C++ and Fortran code
- Useful linear algebra, Fourier transform, and random number capabilities

Besides its obvious scientific uses, Numpy can also be used as an efficient multi-dimensional container of generic data. Arbitrary data-types can be defined using Numpy which allows Numpy to seamlessly and speedily integrate with a wide variety of databases.

Pandas

Pandas is an open-source Python Library providing high-performance data manipulation and analysis tool using its powerful data structures. Python was majorly used for data munging and preparation. It had very little contribution towards data analysis. Pandas solved this problem. Using Pandas, we can accomplish five typical steps in the processing and analysis of data, regardless of the origin of data load, prepare, manipulate, model, and analyze. Python with Pandas is used in a wide range of fields including academic and commercial domains including finance, economics, Statistics, analytics, etc.

Matplotlib

Matplotlib is a Python 2D plotting library which produces publication quality figures in a variety of hardcopy formats and interactive environments across platforms. Matplotlib can be used in Python scripts, the Python and IPython shells, the Jupyter Notebook, web application servers, and four graphical user interface toolkits. Matplotlib tries to make easy things easy and hard things possible. You can generate plots, histograms, power spectra, bar charts, error charts, scatter plots, etc., with just a few lines of code. For examples, see the sample plots and thumbnail gallery.

For simple plotting the pyplot module provides a MATLAB-like interface, particularly when combined with IPython. For the power user, you have full control of line styles, font properties, axes properties, etc, via an object oriented interface or via a set of functions familiar to MATLAB users.

Scikit – learn

Scikit-learn provides a range of supervised and unsupervised learning algorithms via a consistent interface in Python. It is licensed under a permissive simplified BSD license and is distributed under many Linux distributions, encouraging academic and commercial use.

Python

Python is an interpreted high-level programming language for general-purpose programming. Created by Guido van Rossum and first released in 1991, Python has a design philosophy that emphasizes code readability, notably using significant whitespace.

Python features a dynamic type system and automatic memory management. It supports multiple programming paradigms, including object-oriented, imperative, functional and procedural, and has a large and comprehensive standard library.

- Python is Interpreted – Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- Python is Interactive – you can actually sit at a Python prompt and interact with the interpreter directly to write your programs.

Python also acknowledges that speed of development is important. Readable and terse code is part of this, and so is access to powerful constructs that avoid tedious repetition of code. Maintainability also ties into this may be an all but useless metric, but it does say something about how much code you have to scan, read and/or understand to troubleshoot problems or tweak behaviors. This speed of development, the ease with which a programmer of other languages can pick up basic Python skills and the huge standard library is key to another area where Python excels.

All its tools have been quick to implement, saved a lot of time, and several of them have later been patched and updated by people with no Python background - without breaking.

5.9 Install Python Step-by-Step in Windows and Mac :

Python a versatile programming language doesn't come pre-installed on your computer devices. Python was first released in the year 1991 and until today it is a very popular high-level programming language. Its style philosophy emphasizes code readability with its notable use of great whitespace.

The object-oriented approach and language construct provided by Python enables programmers to write both clear and logical code for projects. This software does not come pre-packaged with Windows.

How to Install Python on Windows and Mac :

There have been several updates in the Python version over the years. The question is how to install Python? It might be confusing for the beginner who is willing to start learning Python but this tutorial will solve your query. The latest or the newest version of Python is version 3.7.4 or in other words, it is Python 3.

Note: The python version 3.7.4 cannot be used on Windows XP or earlier devices.

Before you start with the installation process of Python. First, you need to know about your System Requirements. Based on your system type i.e. operating system and based processor, you must download the python version. My system type is a Windows 64-bit operating system. So the steps below are to install python version 3.7.4 on Windows 7 device or to install Python 3. [Download the Python Cheatsheet here.](#) The steps on how to install Python on Windows 10, 8 and 7 are divided into 4 parts to help understand better.

Download the Correct version into the system

Step 1: Go to the official site to download and install python using Google Chrome or any other web browser.

OR Click on the following link: <https://www.python.org>

Now, check for the latest and the correct version for your operating system.

Step 2: Click on the Download Tab.

Step 3: You can either select the Download Python for windows 3.7.4 button in Yellow Color or you can scroll further down and click on download with respective to their version. Here, we are downloading the most recent python version for windows 3.7.4

Step 4: Scroll down the page until you find the Files option.

Step 5: Here you see a different version of python along with the operating system.

- To download Windows 32-bit python, you can select any one from the three options: Windows x86 embeddable zip file, Windows x86 executable installer or Windows x86 web-based installer.

- To download Windows 64-bit python, you can select any one from the three options: Windows x86-64 embeddable zip file, Windows x86-64 executable installer or Windows x86-64 web-based installer.

Here we will install Windows x86-64 web-based installer. Here your first part regarding which version of python is to be downloaded is completed. Now we move ahead with the second part in installing python i.e. Installation

Note: To know the changes or updates that are made in the version you can click on the Release Note Option.

Installation of Python

Step 1: Go to Download and Open the downloaded python version to carry out the installation process.

Step 2: Before you click on Install Now, Make sure to put a tick on Add Python 3.7 to PATH.

Step 3: Click on Install NOW After the installation is successful. Click on Close.

With these above three steps on python installation, you have successfully and correctly installed Python. Now is the time to verify the installation.

Note: The installation process might take a couple of minutes.

Verify the Python Installation

Step 1: Click on Start

Step 2: In the Windows Run Command, type “cmd”.

Step 3: Open the Command prompt option.

Step 4: Let us test whether the python is correctly installed. Type python –V and press Enter.

Step 5: You will get the answer as 3.7.4

Note: If you have any of the earlier versions of Python already installed. You must first uninstall the earlier version and then install the new one.

Check how the Python IDLE works

Step 1: Click on Start

Step 2: In the Windows Run command, type “python idle”.

Step 3: Click on IDLE (Python 3.7 64-bit) and launch the program

Step 4: To go ahead with working in IDLE you must first save the file. Click on File > Click on Save

Step 5: Name the file and save as type should be Python files. Click on SAVE. Here I have named the files as Hey World.

Step 6: Now for e.g. **enter print**

5.10 SAMPLE CODE:

```
#importing all required python libraries
from sentence_transformers import SentenceTransformer #loading bert sentence model
from keras_dgl.layers import GraphCNN #loading GNN class
import keras.backend as K
from keras.regularizers import l2
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
import os
import re
from sklearn.metrics import accuracy_score
from keras.utils.np_utils import to_categorical
```

```

from keras.layers import MaxPooling2D
from keras.layers import Dense, Dropout, Activation, Flatten, LSTM, GRU, Bidirectional
from keras.layers import Convolution2D
from keras.models import Sequential, load_model, Model
import pickle
import os
from keras.callbacks import ModelCheckpoint
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt #use to visualize dataset values
from model import *
import warnings
warnings.filterwarnings('ignore')
#loading & displaying news dataset
column_news = ['News ID', 'Category', 'SubCategory', 'Title', 'Abstract', 'URL', 'Title_Entities',
'Abstract_Entities']
column_behaviour = ['Impression_ID', 'User_ID', 'Time', 'History', 'Impressions']
news = pd.read_csv('Dataset/news.tsv', sep='\t', header=None, names=column_news)
news
#loading & displaying behaviour dataset
behaviour = pd.read_csv('Dataset/behaviors.tsv', sep='\t', header=None, names=column_behaviour)
behaviour
#loading & initializing BERT model for news embedding
bert = SentenceTransformer('nli-distilroberta-base-v2')
print("BERT initialization completed")
#function to read each news article and user behaviour to prepare training data
#this training data will contains user_id, behaviour of news click and news article
#all news article with user behaviour will be converted to numeric vector using BERT embedding
if os.path.exists("model/bert.npy"):
    X = np.load("model/bert.npy")
    Y = np.load("model/Y.npy")

```



```

news_title = news['Title'].ravel()
news_title = news_title[0:10000]
else:
    X = []
    Y = []
    for i in range(0, 5000): #loop and read each news article and then clean and create training array with user
behavioour
        uid = behaviour[i, 1]
        impression = behaviour[i, 4]
        arr = impression.split(" ")
        clicked = ""
        non_clicked = ""
        for j in range(len(arr)):
            label = arr[j].split("-")
            data = getNews(label[0].strip())
            data = data.lower()
            data = re.sub('[^a-z]+' , ' ', data)#clean news data
            class_label = int(label[1].strip())
            if class_label == 0: #get user behaviour as clicked or non-clicked
                non_clicked += data+" "
            elif class_label == 1:
                clicked += data+" "
        non_clicked = non_clicked.strip()
        clicked = clicked.strip()
        if len(non_clicked) > 1:
            X.append(uid.strip()+" "+non_clicked)
            Y.append(0)
        if len(clicked) > 1:
            X.append(uid.strip()+" "+clicked)
            Y.append(1)
        print(str(i))
X = np.asarray(X)
Y = np.asarray(Y)
embeddings = bert.encode(X, convert_to_tensor=True)#apply bert on news data to start embedding

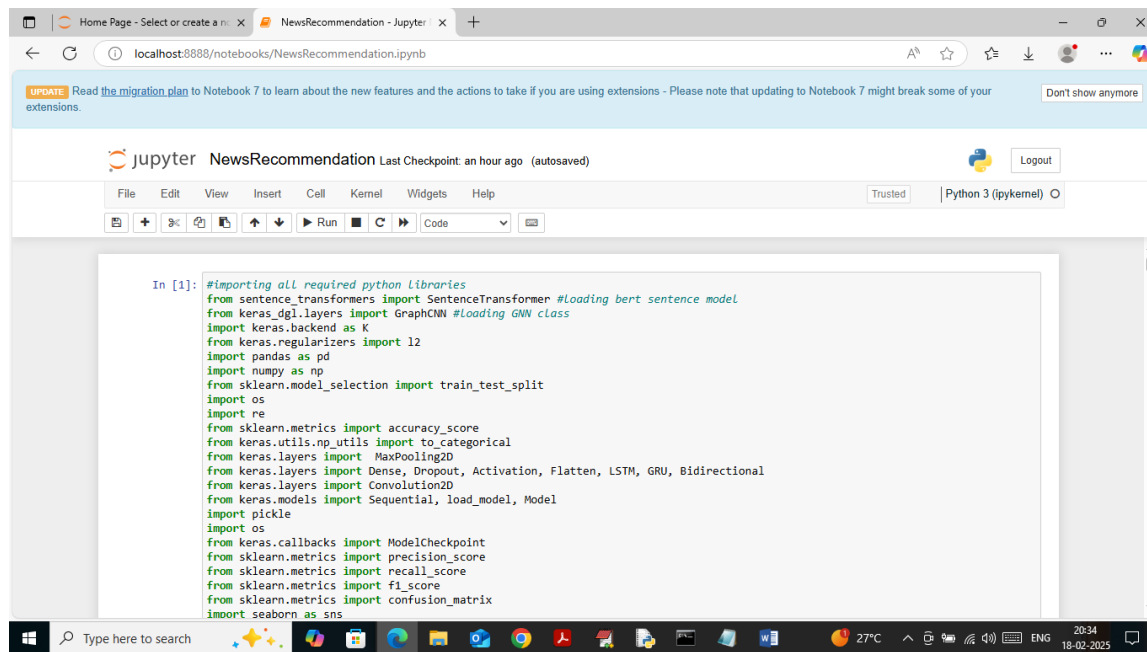
```

```

X = embeddings.numpy()
np.save("model/bert", X)
np.save("model/Y", Y)
print("BERT embedding completed")
print("BERT vector = "+str(X))
#process bert vector by random shuffling
indices = np.arange(X.shape[0])
np.random.shuffle(indices)#shuffling dataset values
X = X[indices]
Y = Y[indices]
news_title = news_title[indices]
Y = to_categorical(Y)#one hot encoding for target binary variable
print("Dataset processing completed")
#split dataset into train and test
X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.2)
print("Dataset Train & Test Split Details")
print("80% dataset used to train algorithms : "+str(X_train.shape[0]))
print("20% dataset used to test algorithms : "+str(X_test.shape[0]))
data = np.load("model/data.npy", allow_pickle=True)
X_train, X_test, y_train, y_test = data
#plot all algorithm performance in tabular format
df = pd.DataFrame(['Graph CNN','Accuracy',accuracy[0]],['Graph CNN','Precision',precision[0]],['Graph
CNN','Recall',recall[0]],['Graph CNN','FSCORE',fscore[0]],
                  ['LSTM','Accuracy',accuracy[1]],['LSTM','Precision',precision[1]],['LSTM','Recall',recall[1]],['LS
TM','FSCORE',fscore[1]],
                  ['CNN','Accuracy',accuracy[2]],['CNN','Precision',precision[2]],['CNN','Recall',recall[2]],['CNN','
FSCORE',fscore[2]],
                  ],columns=['Parameters','Algorithms','Value'])
df.pivot("Parameters", "Algorithms", "Value").plot(kind='bar', figsize=(8, 3))
plt.title("All Algorithms Performance Graph")
plt.show()

```

5.11 OUTPUT SCREENS

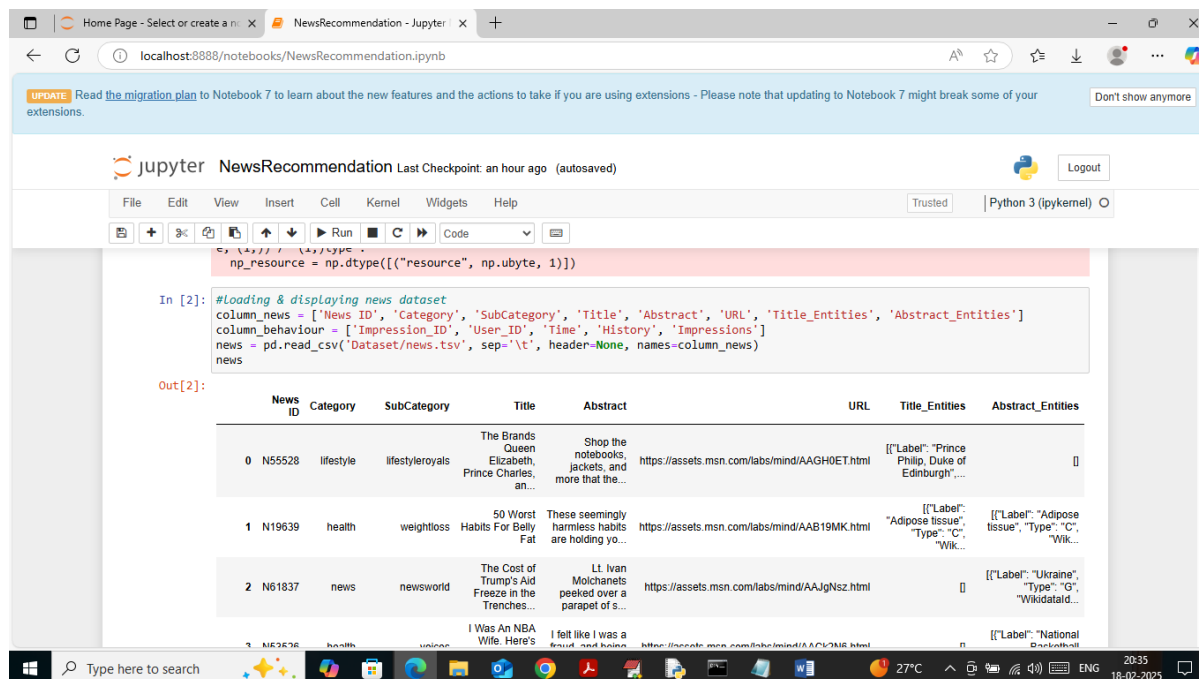


The screenshot shows a Jupyter Notebook interface with the title "NewsRecommendation". The code cell contains the following imports:

```
In [1]: #Importing all required python Libraries
from sentence_transformers import SentenceTransformer #Loading bert sentence model
from keras_dgl.layers import GraphCNN #Loading GNN class
import keras.backend as K
from keras.regularizers import l2
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
import os
import re
from sklearn.metrics import accuracy_score
from keras.utils.np_utils import to_categorical
from keras.layers import MaxPooling2D
from keras.layers import Dense, Dropout, Activation, Flatten, LSTM, GRU, Bidirectional
from keras.layers import Convolution2D
from keras.models import Sequential, load_model, Model
import pickle
import os
from keras.callbacks import ModelCheckpoint
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn.metrics import confusion_matrix
import seaborn as sns
```

Fig 6: Importing Required Python Libraries

In above screen importing required python classes and packages



The screenshot shows the Jupyter Notebook interface with the title "NewsRecommendation". The code cell contains the following code:

```
In [2]: #Loading & displaying news dataset
column_news = ['News_ID', 'Category', 'SubCategory', 'Title', 'Abstract', 'URL', 'Title_Entities', 'Abstract_Entities']
column_behaviour = ['Impression_ID', 'User_ID', 'Time', 'History', 'Impressions']
news = pd.read_csv('Dataset/news.tsv', sep='\t', header=None, names=column_news)
news
```

The output of the code is a table with the following columns: News_ID, Category, SubCategory, Title, Abstract, URL, Title_Entities, and Abstract_Entities. The table displays the first three rows of the dataset.

	News_ID	Category	SubCategory	Title	Abstract	URL	Title_Entities	Abstract_Entities
0	N55528	lifestyle	lifestyleroyals	The Brands Queen Elizabeth, Prince Charles, and...	Shop the notebooks, jackets, and more that the...	https://assets.msn.com/labs/mind/AAGHOET.html	[{"Label": "Prince Philip, Duke of Edinburgh", "..."}]	[]
1	N19639	health	weightloss	50 Worst Habits For Belly Fat	These seemingly harmless habits are holding yo...	https://assets.msn.com/labs/mind/AAB19MK.html	[{"Label": "Adipose tissue", "Type": "C", "Wik...", "Wik..."}]	[{"Label": "Adipose tissue", "Type": "C", "Wik...", "Wik..."}]
2	N61837	news	newsworld	The Cost of Trump's Aid Freeze in the Trenches...	Lt. Ivan Molchanets peeked over a parapet of s...	https://assets.msn.com/labs/mind/AAJgNsZ.html	[{"Label": "Ukraine", "Type": "G", "Wik...", "Wik..."}]	[{"Label": "Ukraine", "Type": "G", "Wik...", "Wik..."}]

Fig 7: Loading and Displaying News Dataset

In above screen loading and displaying MIND news dataset

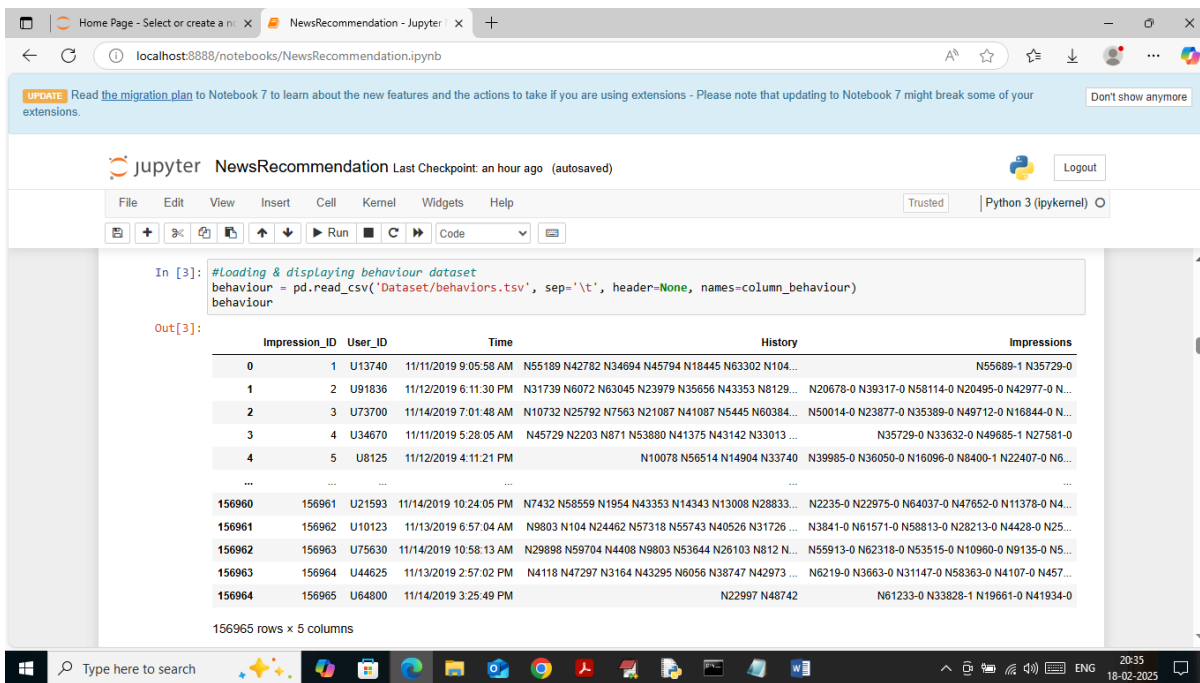


Fig 8: Loading and Displaying User Behavior Dataset

In above screen loading and displaying user behaviors dataset which contains binary label as Clicked or non-clicked

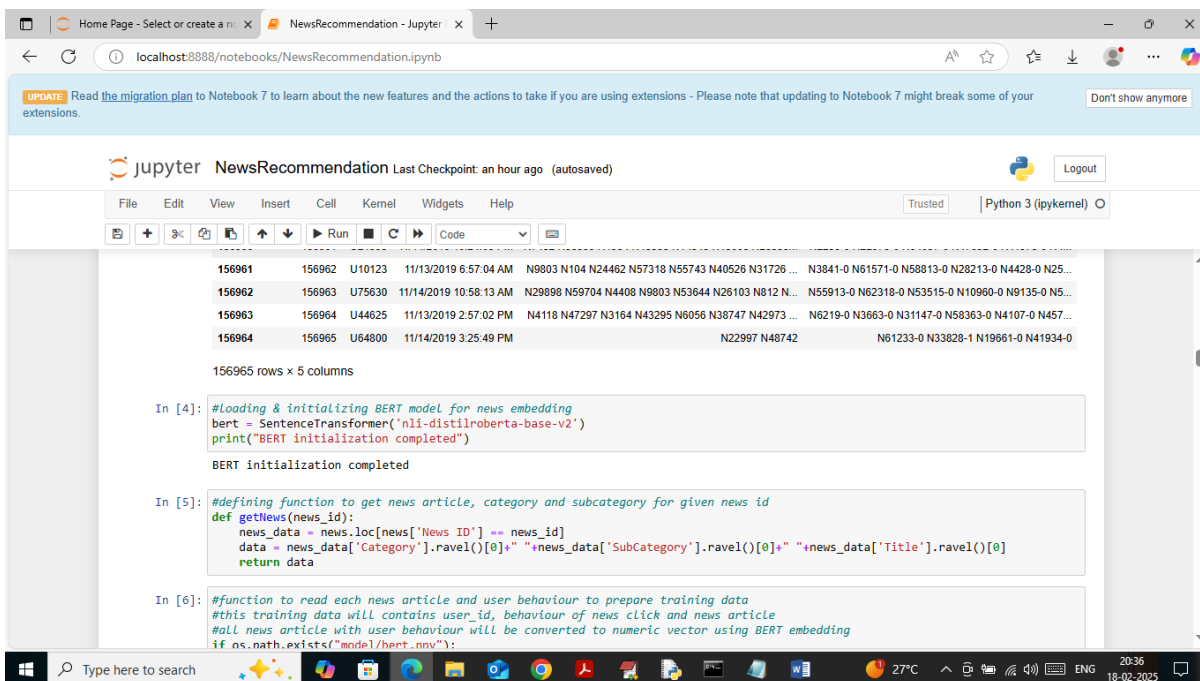
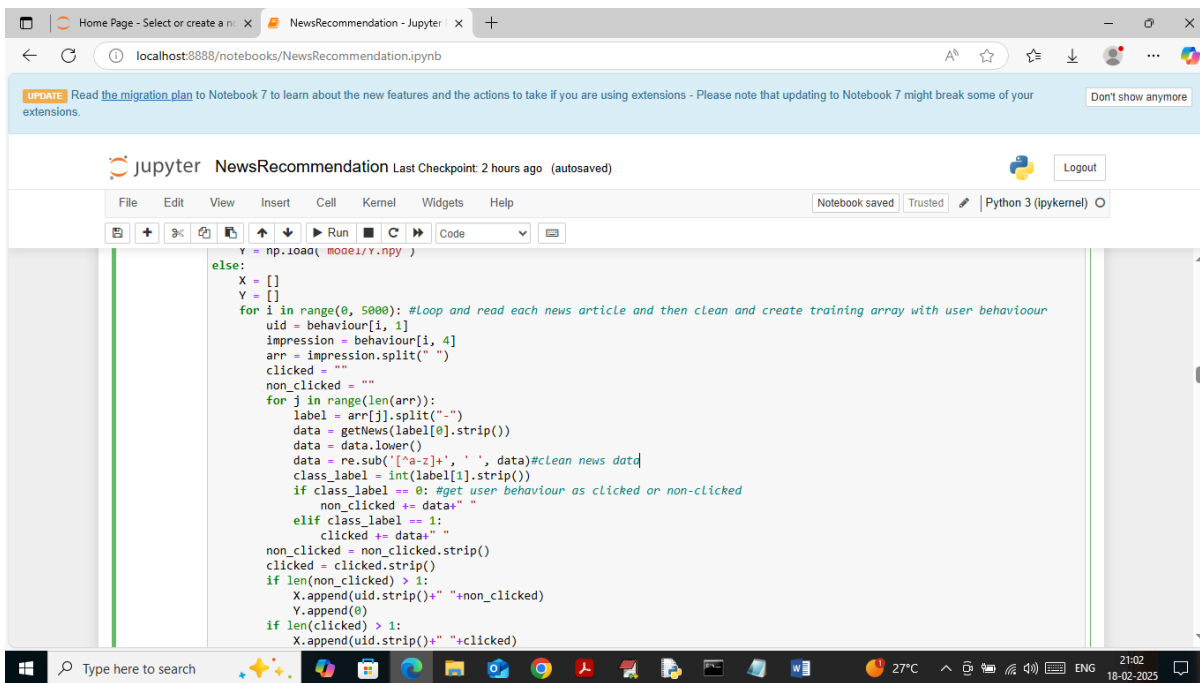


Fig 9: BERT Model Initialization and Data Extraction Function

In above screen initializing and loading BERT model object and then defining function to extract Category, Sub-category and news articles from given news id

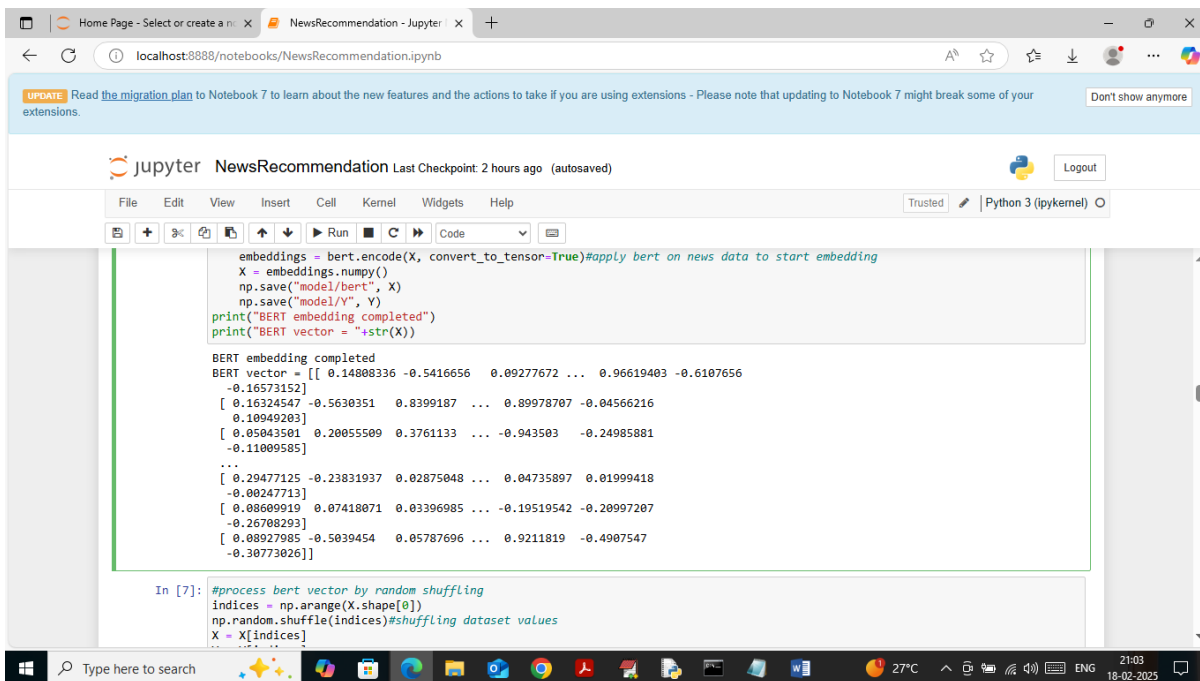


The screenshot shows a Jupyter Notebook interface with the title 'NewsRecommendation'. The code in the cell is as follows:

```
Y = np.load('model/Y.npy')
else:
    X = []
    Y = []
    for i in range(0, 5000): #Loop and read each news article and then clean and create training array with user behaviour
        uid = behaviour[i, 1]
        impression = behaviour[i, 4]
        arr = impression.split(" ")
        clicked = ""
        non_clicked = ""
        for j in range(len(arr)):
            label = arr[j].split(".")
            data = getNews(label[0].strip())
            data = data.lower()
            data = re.sub('[^a-z]+', ' ', data)#clean news data
            class_label = int(label[1].strip())
            if class_label == 0: #get user behaviour as clicked or non-clicked
                non_clicked += data + " "
            elif class_label == 1:
                clicked += data + " "
        non_clicked = non_clicked.strip()
        clicked = clicked.strip()
        if len(non_clicked) > 1:
            X.append(uid.strip()+" "+non_clicked)
            Y.append(0)
        if len(clicked) > 1:
            X.append(uid.strip()+" "+clicked)
```

Fig 10: Data Processing and Training Data Preparation

In above screen reading each news data and then cleaning and creating training array and then embed to BERT model to generated vector and then will get below page



The screenshot shows a Jupyter Notebook interface with the title 'NewsRecommendation'. The code in the cell is as follows:

```
embeddings = bert.encode(X, convert_to_tensor=True)#apply bert on news data to start embedding
X = embeddings.numpy()
np.save("model/bert", X)
np.save("model/Y", Y)
print("BERT embedding completed")
print("BERT vector = "+str(X))

BERT embedding completed
BERT vector = [[ 0.14088336 -0.5416656  0.09277672 ...  0.96619403 -0.6107656
 -0.16573152]
 [ 0.16324547 -0.5630351  0.8399187 ...  0.8978707 -0.04566216
  0.10949203]
 [ 0.05043501  0.20055509  0.3761133 ...  0.943503 -0.24985881
 -0.11009585]
 ...
 [ 0.29477125 -0.23831937  0.02875048 ...  0.04735897  0.01999418
 -0.00247713]
 [ 0.08609919  0.07418071  0.03396985 ... -0.19519542 -0.20997207
 -0.26708293]
 [ 0.08927985 -0.5039454  0.05787696 ...  0.9211819 -0.4907547
 -0.30773026]]

In [7]: #process bert vector by random shuffling
indices = np.arange(X.shape[0])
np.random.shuffle(indices)#shuffling dataset values
X = X[indices]
```

Fig 11: Generated BERT Embedding Vectors

In above screen BERT vector generated from all news data

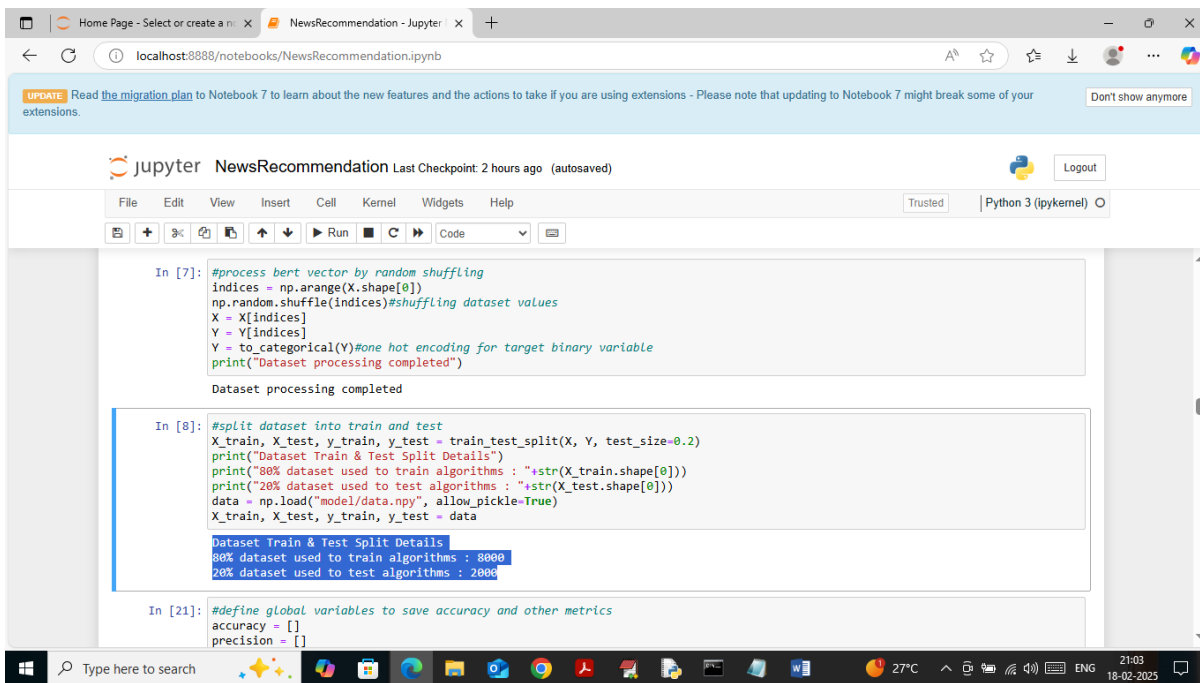


Fig 12: Dataset Shuffling and Train-Test Split

In above screen shuffling and processing BERT vector and then splitting BERT vector into train and test where application using 80% vector for training and 20% for testing and then can see train and test data size

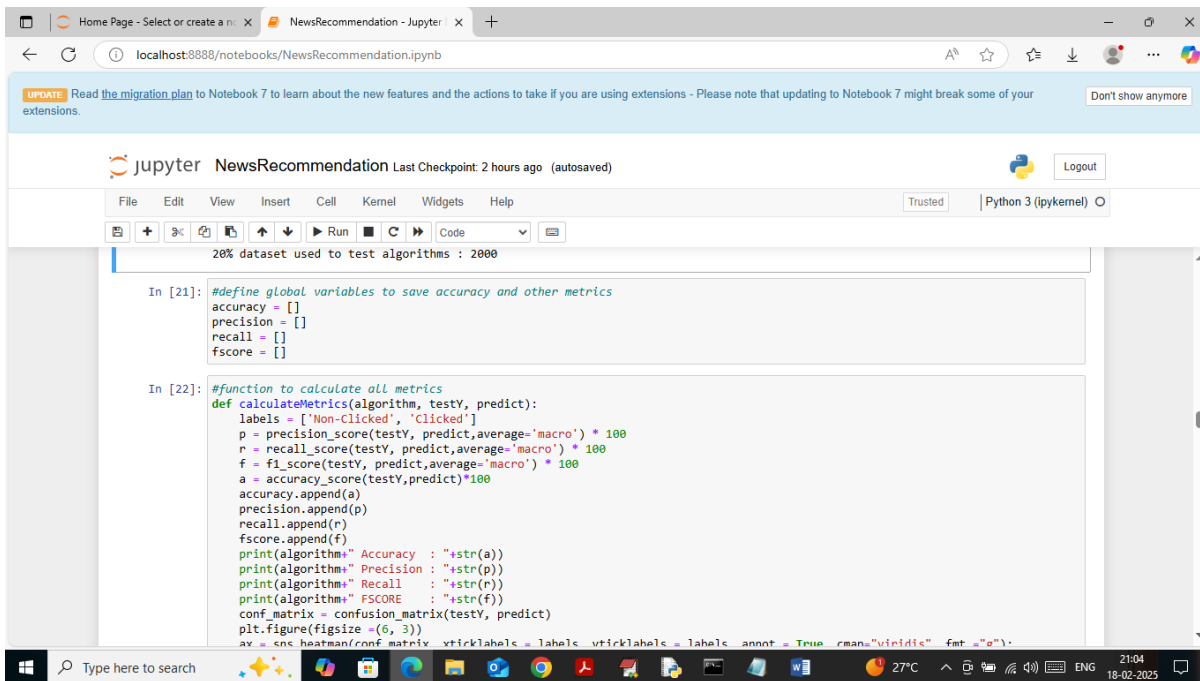


Fig 13: Performance Metrics Calculation Setup

In above screen defining function to calculate accuracy and other metrics

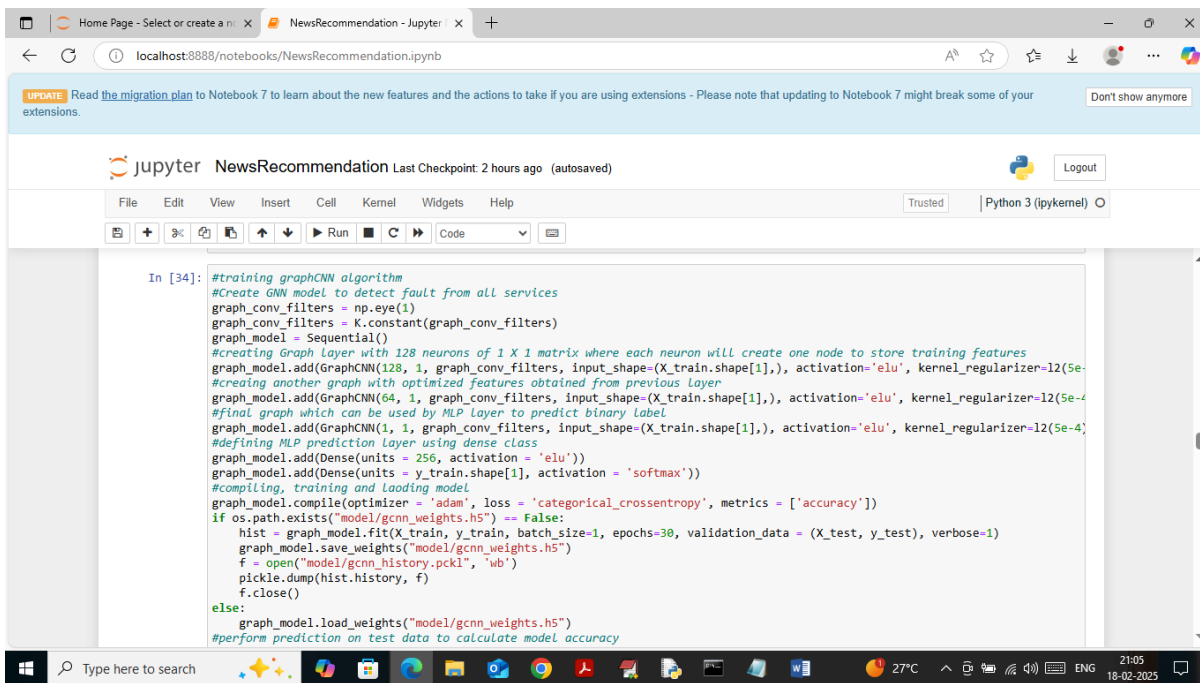


Fig 14: Graph CNN Model Training Process

In above screen defining architecture of GCNN algorithm and then input 80% data to train a model and this model will be applied on 20% test data to calculate prediction accuracy and then will get below page

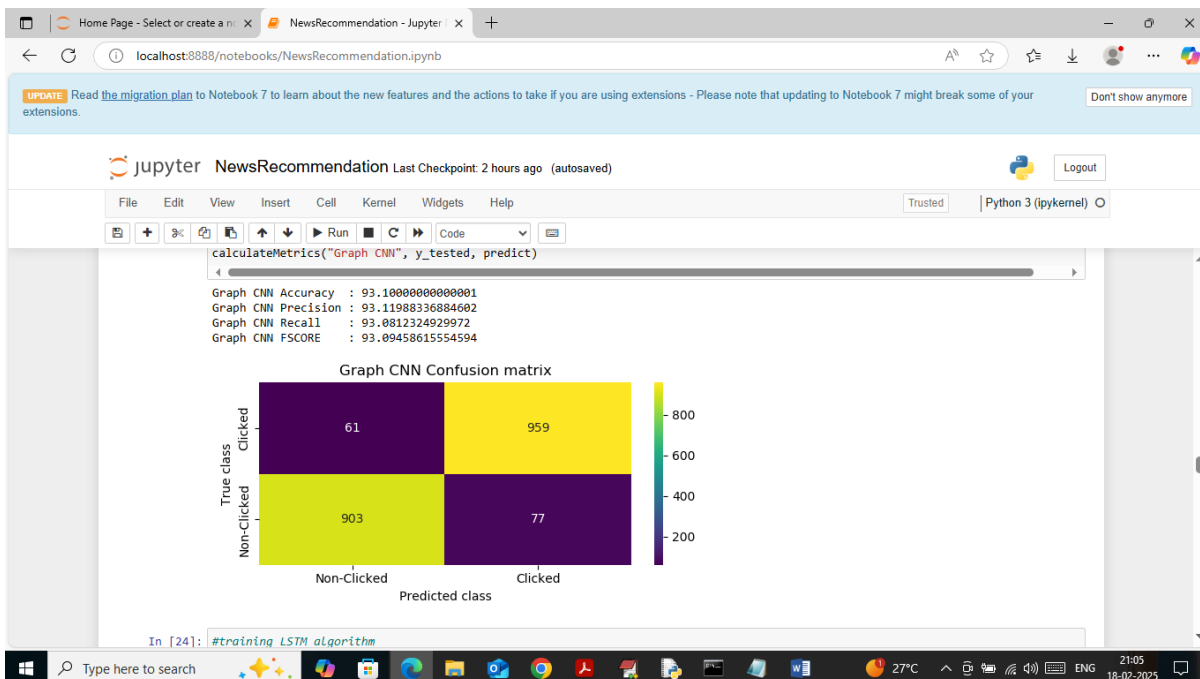
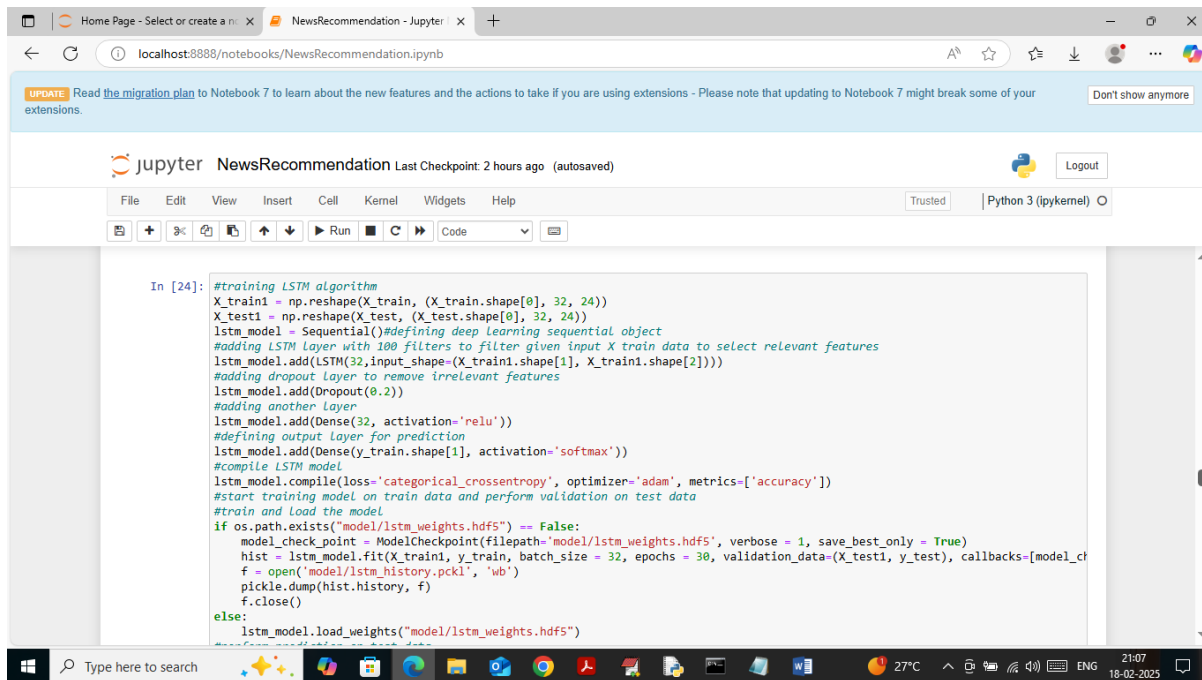


Fig 15: Graph CNN Performance Results (Accuracy & Confusion Matrix)

In above screen GCNN algorithm got 93% accuracy on test data can see other metrics like precision, recall and FSCORE. In above confusion matrix graph x-axis represents Predicted Labels and y-axis represents true labels

and then yellow boxes in diagonal represents correct prediction count and blue boxes represents incorrect prediction count which are very few



The screenshot shows a Jupyter Notebook interface with a code cell containing the following Python code for training an LSTM model:

```
In [24]: #training LSTM algorithm
X_train1 = np.reshape(X_train, (X_train.shape[0], 32, 24))
X_test1 = np.reshape(X_test, (X_test.shape[0], 32, 24))
lstm_model = Sequential()#defining deep learning sequential object
#adding LSTM Layer with 100 filters to filter given input X train data to select relevant features
lstm_model.add(LSTM(32,input_shape=(X_train1.shape[1], X_train1.shape[2])))
#adding dropout Layer to remove irrelevant features
lstm_model.add(Dropout(0.2))
#adding another Layer
lstm_model.add(Dense(32, activation='relu'))
#defining output Layer for prediction
lstm_model.add(Dense(y_train.shape[1], activation='softmax'))
#compile LSTM model
lstm_model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
#start training model on train data and perform validation on test data
#train and load the model
if os.path.exists("model/lstm_weights.hdf5") == False:
    model_checkpoint = ModelCheckpoint(filepath="model/lstm_weights.hdf5", verbose = 1, save_best_only = True)
    hist = lstm_model.fit(X_train1, y_train, batch_size = 32, epochs = 30, validation_data=(X_test1, y_test), callbacks=[model_checkpoint])
    f = open("model/lstm_history.pkl", 'wb')
    pickle.dump(hist.history, f)
    f.close()
else:
    lstm_model.load_weights("model/lstm_weights.hdf5")
```

Fig 16: LSTM Model Training Process

In above screen training LSTM algorithm and below is the LSTM performance output

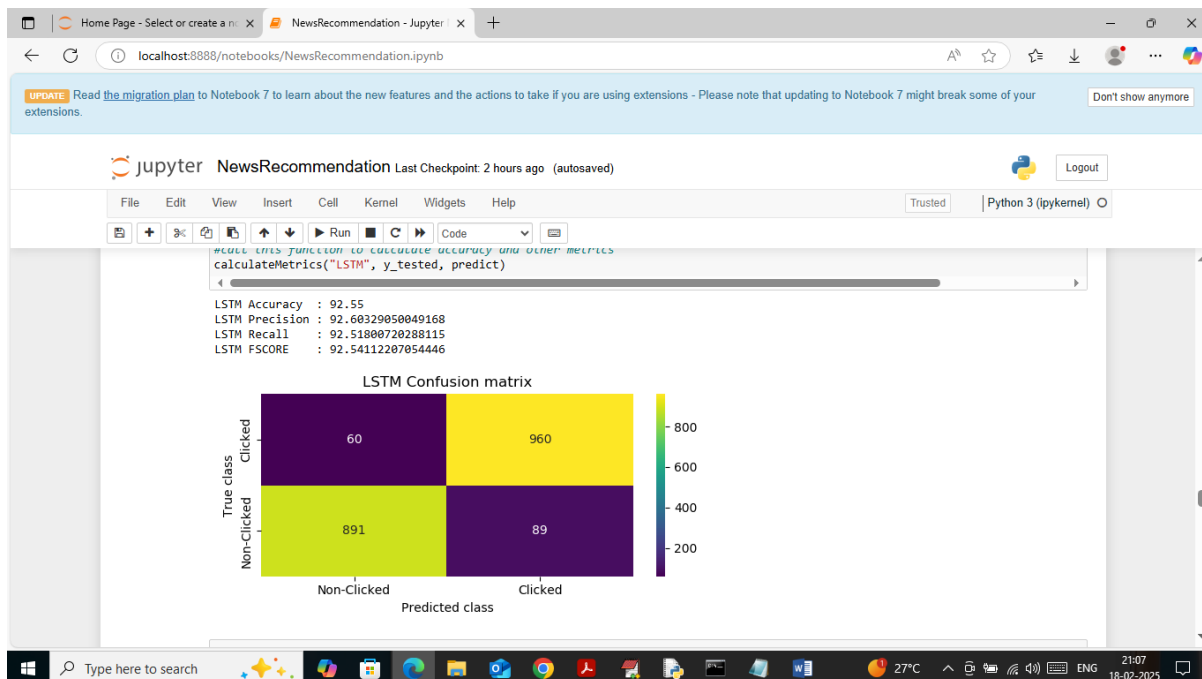
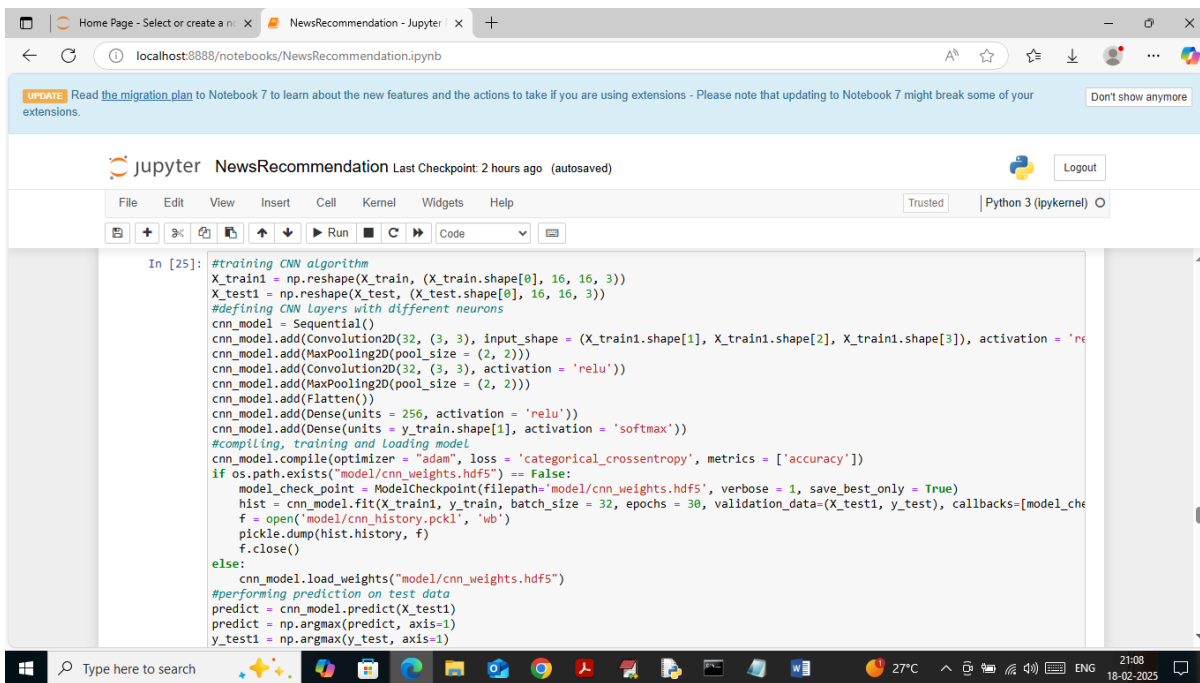


Fig 17: LSTM Model Performance Results

In above screen LSTM got 92% accuracy and can see other metrics also



```
In [25]: #training CNN algorithm
X_train1 = np.reshape(X_train, (X_train.shape[0], 16, 16, 3))
X_test1 = np.reshape(X_test, (X_test.shape[0], 16, 16, 3))
#defining CNN layers with different neurons
cnn_model = Sequential()
cnn_model.add(Convolution2D(32, (3, 3), input_shape = (X_train1.shape[1], X_train1.shape[2], X_train1.shape[3]), activation = 'relu'))
cnn_model.add(MaxPooling2D(pool_size = (2, 2)))
cnn_model.add(Convolution2D(32, (3, 3), activation = 'relu'))
cnn_model.add(MaxPooling2D(pool_size = (2, 2)))
cnn_model.add(Flatten())
cnn_model.add(Dense(units = 256, activation = 'relu'))
cnn_model.add(Dense(units = y_train.shape[1], activation = 'softmax'))
#compiling, training and loading model
cnn_model.compile(optimizer = "adam", loss = 'categorical_crossentropy', metrics = ['accuracy'])
if os.path.exists("model/cnn_weights.hdf5") == False:
    model_checkpoint = ModelCheckpoint(filepath="model/cnn_weights.hdf5", verbose = 1, save_best_only = True)
    hist = cnn_model.fit(X_train1, y_train, batch_size = 32, epochs = 30, validation_data=(X_test1, y_test), callbacks=[model_checkpoint])
    f = open("model/cnn_history.pkl", 'wb')
    pickle.dump(hist.history, f)
    f.close()
else:
    cnn_model.load_weights("model/cnn_weights.hdf5")
#performing prediction on test data
predict = cnn_model.predict(X_test1)
predict = np.argmax(predict, axis=1)
y_test1 = np.argmax(y_test, axis=1)
```

Fig 18: CNN Model Training Process

In above screen training CNN algorithm and below is the performance output

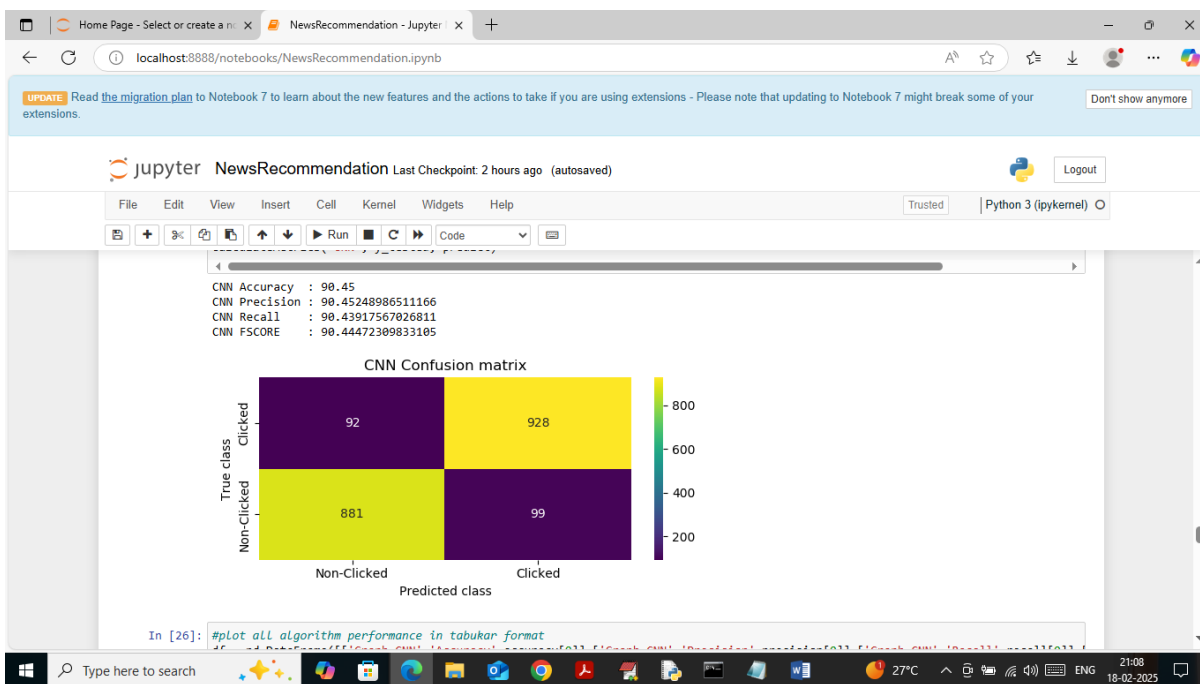


Fig 19: CNN Model Performance Results

In above screen CNN got 90% accuracy and can see other metrics also

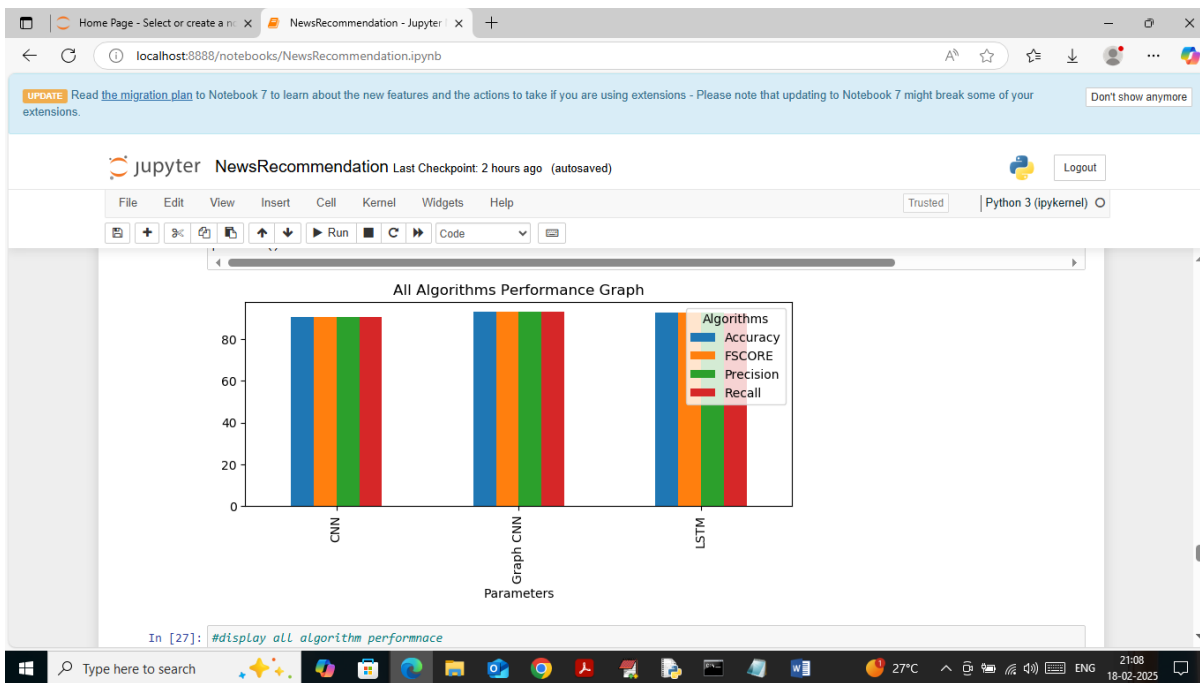


Fig 20: Performance Comparison Graph of All Algorithms

In above screen displaying comparison graph between all algorithms where x-axis represents algorithm names and y-axis represents accuracy and other metrics in different color bars and in all algorithm Graph CNN got high accuracy

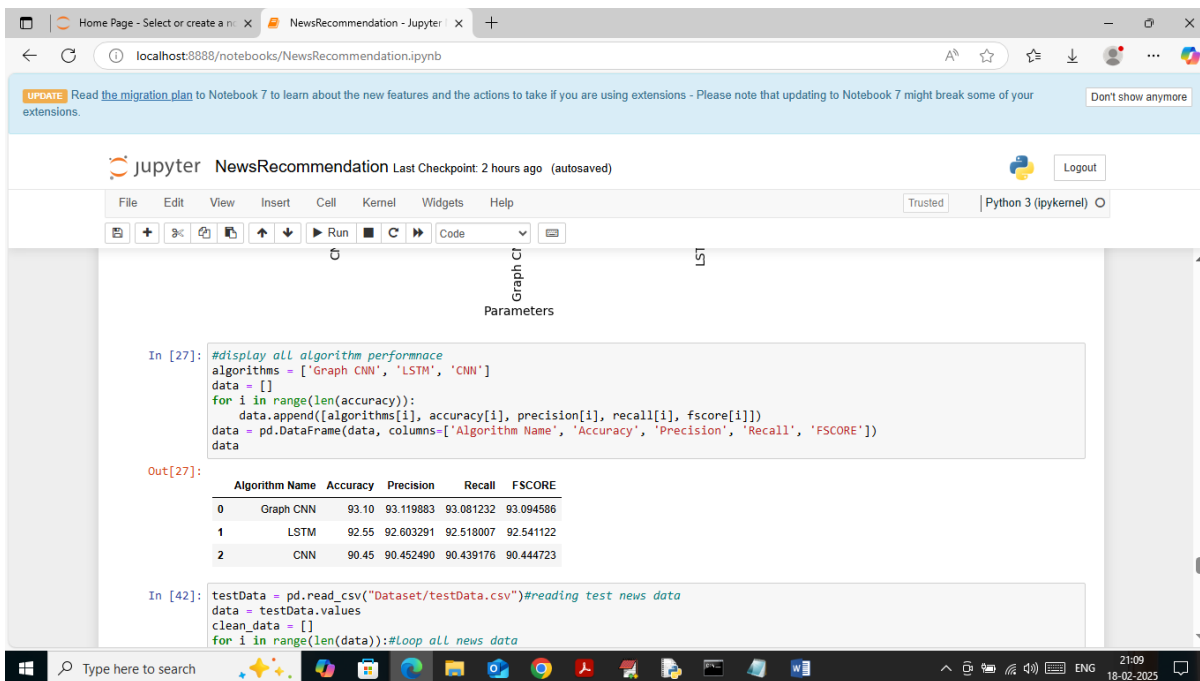


Fig 21: Performance Comparison Table

In above screen displaying comparison table between all algorithms

```

In [42]: testData = pd.read_csv("Dataset/testData.csv")#reading test news data
data = testData.values
clean_data = []
for i in range(len(data)):#Loop all news data
    uid = data[i,0] #get user id
    article = data[i,1] #get news article
    article = article.lower()
    article = re.sub('[^a-z]+', '', article) #clean and process news
    clean_data.append(uid+" "+article.strip())
clean_data = bert.encode(clean_data, convert_to_tensor=True)#apply bert to generate embedding
clean_data = clean_data.numpy()
predict = graph_model.predict(clean_data, batch_size=1)#apply GNN to predict user behaviour to predict click or no-click
predict = np.argmax(predict, axis=-1)
for i in range(len(predict)): #Loop all prediction and print them
    print("User ID = "+uid)
    print("News = "+data[i,1])
    if predict[i] == 0:
        print("User Will not CLICK =====> News Not Recommended")
        print()
    if predict[i] == 1:
        print("User Will CLICKED =====> News Recommended")
        print()

```

Fig 22: Input Test Data for Recommendation Prediction

In above screen reading test news data along with user id and then applying Graph CNN algorithm to predict user behavior and below is the prediction output

```

print()
User ID = U8355
News = sports football nfl videos Every touchdown from every Week 10 game
User Will not CLICK =====> News Not Recommended

User ID = U8355
News = news newsworld U.K. Politicians Fight First December Election in Almost a Century
User Will not CLICK =====> News Not Recommended

User ID = U8355
News = sports football nfl Charles Rogers, former Michigan State football, Detroit Lions star, dead at 38 Charles Rogers, the former Michigan State football star whom the Detroit Lions selected with the second overall pick in 2003 NFL draft, has died
User Will CLICKED =====> News Recommended

User ID = U8355
News = news newsworld Porsche launches into second story of New Jersey building, killing 2 The Porsche went airborne off a median in Toms River, causing it to crash into a red brick building.
User Will CLICKED =====> News Recommended

User ID = U8355
News = sports football_ncaa Colorado's buffalo mascot, Ralpie V, gets the boot for being too fast Ralpie V's 12-year tenure as Colorado's live mascot is coming to an end due to safety concerns. Ralpie was running too fast for her handlers
User Will CLICKED =====> News Recommended

```

In above output screen in first two lines displaying user id along with news data and in 3rd line displaying predicted value whether user will click or not news article and based on click news will be recommended.

6. SYSTEM TEST

6.1 INTRODUCTION

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub assemblies, assemblies and/or a finished product. It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement.

6.2 TYPES OF TESTS

6.2.1 Unit testing

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application. It is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

6.2.2 Integration Testing

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfactory, as shown by successful unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

6.2.3 Functional Testing

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

Valid Input : identified classes of valid input must be accepted.

Invalid Input : identified classes of invalid input must be rejected.

Functions : identified functions must be exercised.

Output : identified classes of application outputs must be exercised.

Systems/Procedures : interfacing systems or procedures must be invoked.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined.

6.2.4 System Test

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

6.2.5 White Box Testing

White Box Testing is a testing in which in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose. It is purpose. It is used to test areas that cannot be reached from a black box level.

6.2.6 Black Box Testing

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document, such as specification or requirements document. It is a testing in which the software under test is treated, as a black box .you cannot “see” into it. The test provides inputs and responds to outputs without considering how the software works.

6.2.7 Unit Testing

Unit testing is usually conducted as part of a combined code and unit test phase of the software lifecycle, although it is not uncommon for coding and unit testing to be conducted as two distinct phases.

Test strategy and approach

Field testing will be performed manually and functional tests will be written in detail.

Test objectives

- All field entries must work properly.
- Pages must be activated from the identified link.
- The entry screen, messages and responses must not be delayed.

Features to be tested

- Verify that the entries are of the correct format
- No duplicate entries should be allowed

- All links should take the user to the correct page.

6.2.8 Integration Testing

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects.

The task of the integration test is to check that components or software applications, e.g. components in a software system or – one step up – software applications at the company level – interact without error.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

6.2.9 Acceptance Testing

User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

Test cases1:

Test case for Login form:

FUNCTION:	LOGIN
EXPECTED RESULTS:	Should Validate the user and check his existence in database
ACTUAL RESULTS:	Validate the user and checking the user against the database
LOW PRIORITY	No
HIGH PRIORITY	Yes

Test case2:**Test case for User Registration form:**

FUNCTION:	USER REGISTRATION
EXPECTED RESULTS:	Should check if all the fields are filled by the user and saving the user to database.
ACTUAL RESULTS:	Checking whether all the fields are field by user or not through validations and saving user.
LOW PRIORITY	No
HIGH PRIORITY	Yes

Test case3:**Test case for Change Password:**

When the old password does not match with the new password ,then this results in displaying an error message as “ OLD PASSWORD DOES NOT MATCH WITH THE NEW PASSWORD”.

FUNCTION:	Change Password
EXPECTED RESULTS:	Should check if old password and new password fields are filled by the user and saving the user to database.
ACTUAL RESULTS:	Checking whether all the fields are field by user or not through validations and saving user.
LOW PRIORITY	No
HIGH PRIORITY	Yes

7. CONCLUSION

The implementation of a news recommendation system using Graph Convolutional Neural Networks (GCNs) presents a powerful approach to delivering personalized and context-aware news content. By representing users, articles, and their interactions as a graph, the system effectively captures complex relationships and higher-order connectivity that traditional recommendation methods often overlook. This enables more accurate and relevant recommendations, addressing challenges such as data sparsity and the cold-start problem.

The proposed system's ability to incorporate diverse information—including semantic, temporal, and behavioral data—helps improve recommendation diversity, novelty, and timeliness, enhancing user satisfaction and engagement. Moreover, the use of GCNs allows for scalable and real-time updates, making the system practical for dynamic news environments where content and user interests continually evolve.

Despite the computational and design complexities involved in constructing and maintaining the graph-based model, the benefits in terms of recommendation quality and adaptability make this approach highly valuable. With appropriate system integration, privacy safeguards, and ongoing maintenance, this GCN-powered recommendation system has the potential to significantly improve the user experience on news platforms, driving higher retention and engagement.

In summary, this project demonstrates the effectiveness of graph neural networks in modern recommendation systems and lays a solid foundation for future enhancements, such as incorporating richer contextual signals and exploring more advanced graph learning techniques.

8. FUTURE ENHANCEMENTS

Future enhancements of the GCN-based news recommendation system focus on improving personalization, scalability, and user trust. Integrating richer contextual information such as user mood, location, and real-time trends can enable more dynamic and precise recommendations, while incorporating multimodal data like images, videos, and audio can enhance content understanding. Exploring advanced graph models such as Graph Attention Networks (GAT), graph transformers, and dynamic GNNs can better capture complex relationships and evolving user interests. Improving computational efficiency will help scale the system for large datasets. Additionally, enhancing explainability will increase user trust by providing clear reasons for recommendations, while fairness-aware techniques can ensure unbiased and diverse content delivery. Finally, integrating cross-platform data and user feedback with reinforcement learning can further refine and adapt recommendations over time.

REFERENCES

- Ying, R., He, R., Chen, K., Eksombatchai, P., Hamilton, W. L., & Leskovec, J. (2018). Graph Convolutional Neural Networks for Web-Scale Recommender Systems. *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*, 974–983.
- Wu, Z., Pan, S., Chen, F., Long, G., Zhang, C., & Philip, S. Y. (2021). A Comprehensive Survey on Graph Neural Networks. *IEEE Transactions on Neural Networks and Learning Systems*, 32(1), 4-24.
- Kipf, T. N., & Welling, M. (2017). Semi-Supervised Classification with Graph Convolutional Networks. *International Conference on Learning Representations (ICLR)*.
- Zhang, S., Yao, L., Sun, A., & Tay, Y. (2019). Deep Learning Based Recommender System: A Survey and New Perspectives. *ACM Computing Surveys*, 52(1), 1-38.
- Hamilton, W. L., Ying, R., & Leskovec, J. (2017). Inductive Representation Learning on Large Graphs. *Advances in Neural Information Processing Systems (NeurIPS)*, 1024-1034.
- Ying, R., Bourgeois, D., You, J., Zitnik, M., & Leskovec, J. (2019). GNNExplainer: Generating Explanations for Graph Neural Networks. *Advances in Neural Information Processing Systems (NeurIPS)*.
- He, X., Liao, L., Zhang, H., Nie, L., Hu, X., & Chua, T. S. (2017). Neural Collaborative Filtering. *Proceedings of the 26th International Conference on World Wide Web (WWW)*, 173-182.
- Rendle, S., Freudenthaler, C., Gantner, Z., & Schmidt-Thieme, L. (2009). BPR: Bayesian Personalized Ranking from Implicit Feedback. *Proceedings of the 25th Conference on Uncertainty in Artificial Intelligence (UAI)*, 452–461.
- Hamilton, W., Ying, Z., & Leskovec, J. (2017). Representation Learning on Graphs: Methods and Applications. *IEEE Data Engineering Bulletin*, 40(3), 52-74.
- Perozzi, B., Al-Rfou, R., & Skiena, S. (2014). DeepWalk: Online Learning of Social Representations. *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 701-710.